

計算機実験ハンドブック

2024 年度版

東京大学理学部物理学科

はじめに

理学部物理学科 3 年の講義「計算機実験 I」および「計算機実験 II」は、理論・実験を問わず、学部～大学院～で必要とされる現代的かつ普遍的な計算機の素養を身につけることを目標としている。基礎的な数値計算アルゴリズムとその応用を学習するだけでなく、実習を通じて、計算機の操作、C 言語を用いたプログラミング技術、 $\text{\LaTeX}$ による科学論文の作成技術などを学ぶ。本冊子は、「計算機実験 I」および「計算機実験 II」の時間に、より効果的に実習を進めることができるよう、必要とされる技術的な点をまとめたものである。なお、実習に必要な環境の準備については、「計算機実習のための環境整備」(<https://utphys-comp.github.io>) にまとめてあるので、そちらを参照してほしい。また、教育用計算機システム ECCS の端末を用いる場合は、<https://utelecon.adm.u-tokyo.ac.jp/oc/#eccs> にある情報を参照のこと。

改訂履歴

2024 年度版 (2024 年 9 月)

- 例 2.3.2 のバグを修正
- ECCS 端末に関する情報の URL を更新
- `man` コマンドがインストールされていない場合についての注釈を追加

2023 年度版 (2023 年 9 月)

- 例 2.9.1 のバグを修正
- C 言語のフォーマットを変更

2022 年度版 (2022 年 9 月)

- SSH 鍵転送に関する記述を追加
- C 言語入門を `gnu11` ベースに大改訂
- $\text{\LaTeX}$ を `uplatex` ベースに変更
- `latexmk` と `lualatex` に対する記述を追加

2021 年度第 2 版 (2021 年 9 月)

- 「1.1.1 コマンドと引数」を追加
- 「1.1.3 オンラインマニュアル」に `-h`, `-help` オプションの説明を追加
- 「2.6 複素数」を追加

2021 年度版 (2021 年 4 月)

- 「計算機実習のための環境整備」に関する記述等を追加

2019 年度版 (2019 年 9 月)

- SSH の公開鍵認証についての記述を追加
- Gnuplot のファイルへの出力、 $\text{\LaTeX}$ への図の取り込みを EPS 形式から PDF 形式に変更
- 動的な二次元配列についての記述を追加

2018 年度版 (2018 年 9 月)

- Gnuplot についての解説を充実
- バージョン管理システムの章を別ファイルに

2017 年度版 (2017 年 4 月)

- 名称を「計算機実験ハンドブック」に変更
- バージョン管理システムの中の URL を修正

2016 年度版 (2016 年 4 月)

- ECCS2016 にあわせて改訂

2015 年度版 (2015 年 4 月)

- 改訂新版
- 付録にバージョン管理システムを追加

目次

第 1 章	UNIX 入門	5
1.1	UNIX のコマンド	5
1.1.1	コマンドと引数	5
1.1.2	ファイルの操作	6
1.1.3	オンラインマニュアル	11
1.1.4	プロセスの管理とトラブル時の対策	12
1.1.5	パイプとリダイレクション	13
1.1.6	コマンド履歴とコマンドライン補完	14
1.1.7	その他有用なコマンド	15
1.2	リモートログインとファイル転送	17
1.2.1	公開鍵暗号方式	17
1.2.2	鍵の作成と登録	17
1.2.3	リモートログイン	18
1.2.4	多段 SSH	19
1.2.5	ファイルの転送	19
1.3	Emacs を使う	20
1.3.1	チュートリアル	20
1.3.2	編集作業	20
1.3.3	Emacs の様々なコマンド	22
1.4	Gnuplot を使う	22
1.4.1	Gnuplot の起動とデータのプロット	22
1.4.2	スクリプトファイルの読み込み、書き出し	23
1.4.3	プロットスタイル	24
1.4.4	3 次元プロット	25
1.4.5	x 軸・ y 軸の設定	26
1.4.6	表題・凡例・図の形	27
1.4.7	矢印・文字列	27
1.4.8	文字のスタイル	27
1.4.9	PDF ファイルへの出力	28
第 2 章	C 言語入門	29
2.1	C 言語の基礎知識	29
2.1.1	なぜ C 言語?	29
2.1.2	まずはコンパイル	30
2.1.3	書式の慣例	32
2.1.4	コンパイル (2)	33
2.1.5	ライブラリのリンク	34
2.2	制御文	35

2.2.1	Cにおける論理値	35
2.2.2	ブロックについて	36
2.2.3	if 文	36
2.2.4	for 文	37
2.2.5	while 文	38
2.2.6	break 文	38
2.2.7	continue 文	39
2.3	配列	40
2.3.1	1次元配列	40
2.3.2	2次元配列	40
2.4	文字列と標準入力	41
2.4.1	文字列	41
2.4.2	標準入力 (1): <code>gets</code> 関数	42
2.4.3	標準入力 (2): <code>scanf</code> 関数	42
2.5	ポインタ	43
2.5.1	とりあえず (1)	43
2.5.2	とりあえず (2)	44
2.5.3	ポインタと配列	45
2.5.4	<code>const</code> とポインタについて	45
2.6	複素数	46
2.7	関数	47
2.7.1	あまり良くない例	47
2.7.2	関数化	47
2.7.3	ポインタを引数にする関数	48
2.7.4	関数ポインタ	49
2.7.5	Fortran の関数や手続きの利用	49
2.8	構造体	50
2.9	ファイルの取り扱い	51
2.9.1	ファイルから数字の読み込み	51
2.9.2	ファイルへの数字の書き出し	53
2.10	その他の制御文	53
2.10.1	<code>switch-case</code> 文	54
2.10.2	<code>do-while</code> 文	54
2.11	コマンドライン引数の受け渡し	55
2.11.1	ポインタ配列	55
2.11.2	新しい <code>main</code> 関数	56
2.12	動的な配列の確保: <code>malloc</code> と <code>free</code>	58
2.12.1	<code>malloc</code> の使い方	58
2.12.2	<code>free</code> の使い方	59
2.12.3	動的な 2次元配列	60
2.13	<code>segmentation fault</code> が出たら	63
2.14	外部ヘッダの利用	64
2.14.1	外部ヘッダの利用	64
2.14.2	外部ライブラリの利用	65
2.15	外部ツールの利用	66

2.16	練習問題の解答例	67
第 3 章	L^AT_EX 入門	71
3.1	L ^A T _E X の実行	71
3.2	L ^A T _E X に関する全体的なこと	72
3.3	フォント	73
3.4	論文のスタイル	74
	3.4.1 表題	74
	3.4.2 論文の構成	75
3.5	箇条書き	77
3.6	表の作成	78
3.7	数式	79
	3.7.1 添え字	80
	3.7.2 分数	80
	3.7.3 関数	81
	3.7.4 フォント	81
	3.7.5 積分・和・積	82
	3.7.6 根号・ベクトル・上線・チルダ・ハット・時間微分	82
	3.7.7 ギリシャ文字	83
	3.7.8 行列	83
	3.7.9 特殊記号	84
3.8	図の貼り込み	85
	3.8.1 パッケージについて	85
	3.8.2 PDF ファイルの貼り込み	85
	3.8.3 図の回転	86
3.9	参照と参考文献	87
	3.9.1 参照	87
	3.9.2 参考文献	89
	3.9.3 コンパイル	90
	3.9.4 latexmk の利用	90
3.10	L ^A T _E X の種類について	91

第 1 章

UNIX 入門

この章では、UNIX と呼ばれる OS (オペレーティングシステム) の操作法について学ぶ。通常、macOS や Windows では、マウスを使って画面を操作する。実は、macOS の内部では Darwin と呼ばれる UNIX 系の OS が動作しており、「ターミナル」アプリケーションを立ち上げることにより、UNIX の様々な機能に直接アクセスすることができる。また、ワークステーションやスーパーコンピュータなど、より大きな規模の計算を行うことのできる計算機も、ほぼ全て Linux などの UNIX 系 OS である。UNIX 系の OS の特徴としては、シンプル、柔軟かつオープンであり、ネットワークに強いことが挙げられる。シェルやスクリプト言語を組み合わせることにより、簡単なコマンドで複雑な処理を実現することができる。また、ネットワーク経由で使うことが前提となっており、実際に計算機がどこにあるかを意識することなしに、透過的に利用できる。UNIX を取得することにより、目の前の PC だけでなく世界中の計算機を利用することが可能となり、計算能力が一気に増えることになる。

1.1 UNIX のコマンド

この節での操作はすべて、ターミナル (「シェル」とも呼ぶ) 内でコマンドを打ち込み、最後にリターンキーを押すことにより実行する。下線を引いた部分がキーボードからの入力である。行頭の “\$” はプロンプトと呼ばれる。一般にプロンプトは “user@host:~\$” のようにユーザ名やホスト名、カレントディレクトリを表す長い文字列であることが多いが、本書では省略して単に “\$” と書く^{*1}。プロンプトは UNIX のシェルがユーザからのコマンド入力待ちの状態であることを示すものであり、コマンド入力時にタイプする必要はない。

1.1.1 コマンドと引数

入力されたコマンドを解釈し適切なプログラムを実行するのがシェルの役割である。まずは、date と入力して最後にリターンキーを押してみよう。date という名前のプログラムが実行され、(期待通り) 現在の日時が表示される。

```
$ date
Tue 17 Aug 2021 05:14:47 PM JST
```

コマンドには引数 (ひきすう) を指定することもできる。

^{*1} プロンプトの表示形式は自分でカスタマイズすることもできる。

```
$ echo hello
hello
$ echo this is a pen
this is a pen
```

echo は受け取った文字列を順番に出力するだけのプログラムである。最初の echo がプログラム名、2 番目以降が引数 (パラメータ) としてプログラムに渡される。

コマンド入力は半角スペース (空白) で分割されて解釈される。連続する 2 つ以上のスペースは 1 つのスペースと等価である。

```
$ echo this is a pen
this is a pen
$ echo this is a   pen
this is a pen
```

この例ではどちらも this, is, a, pen の 4 つの文字列がプログラム echo に渡される。スペースも含めて渡したいときは二重引用符等で囲み、一つの長い文字列として渡す。

```
$ echo "white space   matters"
white space   matters
```

1.1.2 ファイルの操作

この節では、ファイルを作ったり、消したりといった基本的な操作を紹介する。ファイルというのは計算機が情報を書き込むための一つの単位である。ファイルには名前を付けることができ、その名前を指定することでファイルを編集したり消したりできる。

ファイルの基本操作

ファイルをコピーするには cp コマンド (copy の略) を用いる。例えば、コマンドラインで

```
$ cp /usr/local/example/circle.c circle.c
```

を実行すると、/usr/local/example という名前のディレクトリにあるファイル circle.c が、カレントディレクトリにコピーされる。/usr/local/example/circle.c の部分がコピー元に対応し、後の circle.c がコピー先を意味する。コピー先がディレクトリの場合には、コピー先のディレクトリにコピー元と同じ名前のファイルが作られる。したがって、上の例は、カレントディレクトリを表す "." を用いて、

```
$ cp /usr/local/example/circle.c .
```

としても同じ結果となる。元とは異なる名前のコピーしたい場合には、

```
$ cp /usr/local/example/circle.c oval.c
```

などとする。同じ名前のファイルがすでに存在する場合には上書きされるので注意せよ。

ここで、ls コマンド (list の略) を実行すると

```
$ ls
Desktop  circle.c
```

と表示され、正しくコピーされたことが分かる。このように、`ls` コマンドを使うことで、どのようなファイルがあるのかを知ることができる。さらに、`ls -l` を実行すると

```
$ ls -l
total 2
drwxr-xr-x  2 s001500 student    512 Apr 16 03:40 Desktop
-rw-r--r--  1 s001500 student    372 Apr 16 03:43 circle.c
```

のように、より詳細な情報^{*2}を得ることができる。この`-l`をオプション(あるいはコマンドラインオプション)と呼ぶ。オプションを指定することでプログラムの動作を変えることができる。`ls` コマンドには他にも様々なオプションが用意されている。`-F` オプションをつければ、ファイルの名前の後にファイルの種別を表す文字をつけてくれる。ディレクトリには `/` が、実行可能ファイルには `*` がつく。`-R` オプションをつければ、カレントディレクトリ以下のすべてのディレクトリの中身を見ることができる。また、`-a` とすると、通常表示されない `.` (ピリオド) から始まるファイルも見ることができるようになる。

次にファイルの名前を変えてみよう。先の `circle.c` を `ring.c` に名前を変更するには、`mv` コマンド (`move` の略) を使う。

```
$ mv circle.c ring.c
```

`ls` コマンドで実際に名前が変更されたか確かめてみよう。

続いて `rm` コマンド (`remove` の略) を用いてファイルを消してみよう。

```
$ rm ring.c
```

一度消したファイルは二度と復旧できないので慎重に実行しなければならない。自信がないなら、

```
$ rm -i ring.c
```

のように`-i` オプションをつけるとよい。ファイルごとに消してよいかどうかの確認をしてくれるので安心である。

さて、ここまでディレクトリという言葉が何度か出てきた。UNIX では、ディレクトリという特殊なファイルを使って、たくさんのファイルを tree 状に管理することができる。ディレクトリとは書類をまとめる書類箱だと思えばよいだろう。ときに大きな書類箱のなかに小さな書類箱が入っていることもあるわけで、ディレクトリの中にディレクトリがあってもいっこうに構わない。

^{*2} 表示の1行目は、左端の `d` が、そのファイルがディレクトリであることを示し、次の `rw` はファイルの所有者が読み取り可能 (`r`)、書き込み可能 (`w`)、実行可能 (`x`) であることを示している。(ディレクトリが読み取り可能であるとは、そのディレクトリにどのようなファイルが入っているかを `ls` コマンドで調べられることをいい、ディレクトリが書き込み可能とは、そのディレクトリに新しくファイルやディレクトリを新しく作ったり、すでにあるファイルなどを消したりできることをいう。さらに、ディレクトリが実行可能であるとは、そのディレクトリ内部のファイルを操作できたり、そのディレクトリに移動するとか、そのディレクトリの名前をパスに含めることができることを意味する。) 次の `r-x` は同じグループに所属する人が、読み取りと実行可能であることを示し、その次の `r-x` はそのほかすべての人が読み取りと実行可能であることを示している。隣のカラムの `s001500` は、そのファイルの所有者が `s001500` であることを示し、次のカラムは所有グループを示している。次の `512` は、そのファイルの大きさが `512` byte であることを示している。`Apr 16 03:40` は、そのファイルに最後に変更を加えた日時を示す。そして最後がファイルの名前である。

ログインして最初にいるディレクトリのことをホームディレクトリと呼ぶ。今いるディレクトリ*³がどこかを知るには、`pwd` コマンド (**p**rint **w**orking **d**irectory の略) を使う。`/home/s001500` と表示されれば ルートディレクトリ*⁴の下の `home` というディレクトリの下、`s001500` というディレクトリにいるということが分かる。

試しに `sample` という ディレクトリを作ってみよう*⁵。

```
$ mkdir sample
```

これで `sample` というディレクトリが作成された。`ls` コマンドで確かめてみよう。次に、今作ったディレクトリに移動してみよう。

```
$ cd sample
```

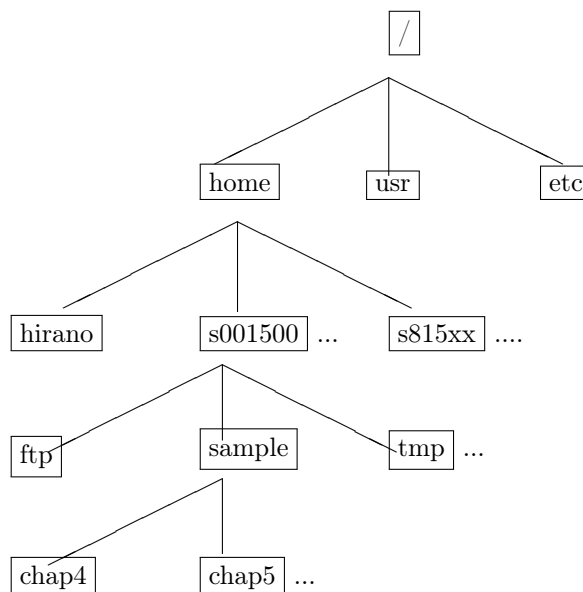
`cd` は **c**hange **d**irectory の略である。ここで、先ほどの `cp` コマンドを使えば、このディレクトリにファイルをコピーすることができる。さらにこの中で `mkdir chap4` と打てば、`sample` ディレクトリの下に `chap4` ディレクトリが作成される。

では、元のディレクトリに戻るにはどうすればよいのだろうか？ それには

```
$ cd ..
```

を実行する。`cd` と `..` の間に空白があることに注意すること。

ツリー構造の中で、どのようにディレクトリやファイルを指定すればよいのだろうか。ディレクトリのツリーを図示すれば次のようになる。



このツリーの中の位置は、一番上の `/` で表されるルートディレクトリから順にたどることより指定することができる。これを「絶対パス」と呼ぶ。あるいは、今いるディレクトリ (カレントディレクトリ) からの相対的な

*³ カレントディレクトリ、あるいはワーキングディレクトリと呼ぶ。

*⁴ ルートディレクトリとは一番上のディレクトリのことであり、最初の `/` がルートディレクトリを表す。

*⁵ すでにあるディレクトリを消すには `rmdir` コマンド (**r**emove **d**irectory の略) を使う。`rmdir` でディレクトリを消すときはそのディレクトリの中にファイルやディレクトリが一つもない状態になっていなくてはならない。消したいディレクトリの中にファイルが残っているときは `rm` コマンドを使い、ディレクトリが残っているときは `rmdir` コマンドを使う。残っているファイルやディレクトリをいちいち消すのが面倒な場合は `rm -r directory` とすれば、`directory` の中に何が残っていてもすべてきれいに消し去ってくれる。

位置で指定する方法もある。これを「相対パス」と呼ぶ。例えば、ホームディレクトリの下 `sample` の下の `chap4` ディレクトリは、絶対的な指定では `/home/s001500/sample/chap4` と書かれるが、カレントディレクトリがホームディレクトリの場合、相対的な指定では `sample/chap4` と書かれる。そのディレクトリの中にあるファイルも `/home/s001500/sample/chap4/circle.c` とか `sample/chap4/circle.c` のように指定することができる。特に一つ上のディレクトリは“`..`”で、カレントディレクトリは“`.`”であらわされる。つまり先ほどの `sample/chap4/circle.c` は `../s001500/sample/chap4/circle.c` と書いても同じものを指す。

これで、自由にディレクトリ間を移動することができるようになった。例えば、すでに使った `ls` コマンドは、`/usr/bin` あるいは `bin` ディレクトリに格納されている*⁶。 `cd /usr/bin` として、そこに移動してから `ls` と打ってみよう。たくさんのファイルが置かれているが、`ls` というファイルは見付かったらどうか*⁷。

なお、`cd` とだけ打てば、いつでも自分のホームディレクトリに戻ってくることができる。ホームディレクトリは `~` (チルダ) とも表されるので `cd ~` としても同じである。他にも特殊な記法として“前のカレントディレクトリ”を表す `-` (ハイフン) が使用できる。

ファイルの中を見る

さて、肝心のファイルの中身を見るにはどうしたらよいのだろう。ファイルの中身を見るにはいくつかの方法がある。ファイルには大別して、中身を見ることができるテキストファイルと中身を見られないバイナリファイルがある。テキストファイルの中身は、`cat` コマンドで見ることができる*⁸。

```
$ cat circle.c
```

`cat` は最も手軽にファイルの中身をのぞくことのできるコマンドである。中身が長すぎる場合、`cat` では流れていってしまうが、

```
$ more circle.c
```

と打つことにより、ちょうどよい所で一旦止めてくれる。`-More-(xx%)` のような表示が左下に見えたら、`スペース` を叩いてみよう。次のページが表示されるはずである。`h` と打つと簡単なヘルプが見られる。`b` か `ctrl-B` で 1 ページ戻る。また `Enter` で 1 行進む。さらに、`/` の後に **文字列** を打つことでファイル中の文字列を検索することができる。検索された文字列を含む行はウィンドウの一番上に表示される。`n` と打てばもう一度同じ文字列を探してくれる。また、`10d` のように数字を入れてから `d` を打てば、その数字分だけ進む。`more` から抜けるには、`q` を入力する。

`more` と似ているが、もう少し使い勝手のよい `less` というコマンドもある。`less` では上向きにスクロールすることができる。`j` で 1 行下に、`k` で 1 行上*⁹に進む。`more` と同じく、`スペース`、`h`、`b`、`d`、`/`、`n`、`q` のようなコマンドも使える。更に `g` でファイルの先頭に、`G` でファイルの最後尾にジャンプすることができる*¹⁰。

*⁶ `which ls` とすることで、`ls` がどこにあるか調べることができる。

*⁷ 例えば、`treasure.here` という名前のファイルを探したい場合、そのファイルがありそうなディレクトリよりも上のディレクトリに行って、`find . -name treasure.here -print` とする。`find` の直後の `.` (ピリオド) は「そのファイルがありそうなディレクトリより上のディレクトリのパス」を指定する。またファイル名があやふやな場合でも、`find . -name 'tre*.here' -print` のように `*` を任意の文字列として合致するファイルを表示してくれる。`find` はとても多機能である。例えば `find . -name 'tre*.here' -exec cat {} \;` のように、探し出したファイルに対してコマンドを実行することもできる。`-exec` 以下の書式は コマンドの引数となるファイル名が `{}` となるように書き、最後に `\;` を加えればよい。`-exec` の代わりに `-ok` を使えばコマンドを実行する前に一々聞いて来るので `rm` などを `find` で実行したいときには安心である。`find . -ctime -1 -print` のように `-ctime` で最終変更期日が 1 日前以内のものだけ表示させるようなことも可能である。`-ctime` の引数を `+1` にすると最終変更期日が 1 日以上前のものだけが表示される。

*⁸ バイナリファイルの中身を覗くための `od (octal dump)` というコマンドもある。

*⁹ UNIX 標準エディタ `vim` のキー操作と同じ。

*¹⁰ `tail circle.c` のようにすれば、ファイルの最後だけ見ることができる。また `tail -30` のように指定することで最後から 30 行目以降を見ることができる。

次に、たくさんあるファイルの中に特定の文字列が含まれているか調べてみよう。

```
$ grep main *.c
```

と入力してみよう。どの“.c”で終るファイルに main という文字列が含まれているか一目瞭然である*¹¹。

ファイルの保護

他の人に見られたくないファイルにはプロテクションをかけることができる。

```
$ chmod o-r circle.c
```

とすれば自分と、自分と同じグループに属する人以外はそのファイルを見ることができなくなる*¹²。

```
$ chmod g-r circle.c
```

とすれば同じグループの人からも見られなくなる。同じ要領で

```
$ chmod o-rwx circle.c
```

とすれば他の人は circle.c というファイルを読むことも書き込むことも実行することもできなくなり、

```
$ chmod go-rwx sample
```

とすれば自分以外のだれも sample というディレクトリにアクセスできなくなる。もし大切なファイルを間違いないなどで変更したくない場合は、

```
$ chmod u-w circle.c
```

のようにすれば、自分自身の書き込みからもファイルを守ることができる*¹³。

ファイルにプロテクションをかける方法がわかったところで、セキュリティの面から見たファイルの保護について説明しておこう。まず、特別な事情がないかぎり、ファイルに user 以外の書き込み許可を出してはならない。次に user 以外には読み取り許可 (および実行許可) も出してはいけないディレクトリやファイルとしては、.Xauthority、.ssh などがある*¹⁴。

ファイルのアクセス許可について不安がある場合は、.bashrc などのファイルに umask 077 の 1 行を入れておくとうい。こうしておく、それ以後のログインで作られるファイルは、user 以外のアクセス許可がいない状態で作られる。その後、許可を出してもよいと判断したファイルに限って、chmod コマンドで許可を出すようにするとよいだろう。

*¹¹ さらに grep -n printf *.c としてみると、各ファイルの何行目に printf という文字列があるのかが分かる。さらに、grep -n -e '.*gram.*' *.c で、.* のところに任意の文字列があてはまるすべての文字列について検索する。シングルクォーテーションで囲まなくてはならない点に注意すること。

*¹² o は other の頭文字で、他の人にも見えるようにするには o-r を o+r に変えて実行する。chmod u=rwx,g+x,o-r circle.c のように指定することもできる。そのほか g (group) は 同じグループ、u (user) は所有者、a (all) はユーザーも含めた全員を表し、r、w、x はそれぞれ、read, write, execute を表す。

*¹³ また変更しなくなった場合は chmod u+w circle.c とすればよい。

*¹⁴ “.” で始まるファイルは隠しファイルである。ホームディレクトリで ls -a とするとこれらのファイルを見ることができる。

1.1.3 オンラインマニュアル

ここまで様々なコマンドを説明してきたが、これらのコマンドのマニュアルは `man` コマンドを使って、計算機上で見ることができる^{*15}。まずはマニュアルのマニュアルを見てみよう。

```
$ man man
```

と打つと、`Reformatting page. Wait...` とでた後、画面が切り替わって、`man` というコマンドのマニュアルが表示される。この画面を表示しているのは、すでに説明した `less`^{*16}なので、1 ページ前に行ったり、スキップしたりするのも容易である。

ここまでで紹介して来た多くのコマンド^{*17}がマニュアルに登録されている。

```
$ man command
```

のようにして、それぞれのコマンドの詳しい意味を調べてみよう。

正しいコマンド名が分かっているときはよいが、もし「～のようなコマンドはないかな？」とか「たしかこんなコマンドがあったはず」と思ったときは、

```
$ man -k keyword
```

を使う。例えば、なにかディレクトリの操作に関するコマンドに関して調べるには、以下のようにすればよい。

```
$ man -k directory
...
mkdir (1)           - Makes a directory
mkdir (2)           - Creates a directory
mkdirhier (1X)      - makes a directory hierarchy
mkfontdir (1X)      - create fonts.dir file from directory of font files
mklost+found (8)    - Makes a lost\found directory for fsck
mvdirt (1)          - Moves (renames) a directory
pwd (1)             - Displays the pathname of the current (working) directory
rename (2)          - Renames a directory or a file within a file system
rmdir (1)           - Removes a directory
rmdir (2)           - Removes a directory file
...
```

左端がマニュアルのタイトルである。右欄にある大雑把なコマンドの意味を参考に目的のものを探し出す。コマンド名の後に書いてある括弧の数字はマニュアルのセクション番号をあらわす。マニュアルは使う目的や内容に応じてセクションに分かれている。もし同じタイトルのマニュアルが二つのセクションに分かれて置かれている場合、それぞれは

^{*15} 環境によっては、ディスク容量を削減するために `man` コマンドがインストールされていないことがある。その際は、Google などで「linux man command」と検索することで、該当コマンドのマニュアルを見つけることができる。

^{*16} あるいは、環境変数 `PAGER` を設定している場合にはそのページャーが立ち上がる。

^{*17} コマンドに限らず C 言語用のライブラリ関数や設定ファイルの書式などもマニュアルに入っている。

```
$ man 2 rmdir
```

のように `man` コマンドの後にセクション番号を書くことで指定することができる。

また、多くのコマンドでは `-h` オプション、あるいは `-help` オプションを指定するとマニュアルが表示される。

```
$ less -help
```

1.1.4 プロセスの管理とトラブル時の対策

この節では、ここではトラブル時の対策を含んだプロセスの管理について説明する。多少面倒な話なので読み飛ばしておいても構わない。

ターミナル上で何かを実行していて止まらなくなったときに最も有効なのは、`ctrl-c` である。`ctrl-c` は実行しているジョブを強制的に終了する。うまくいけばプロンプトが戻って来るはずである。

それでも止まらなかった場合には、`ctrl-z` を入力してみよう。`ctrl-z` はプロセスを一時的に停止させるだけなので、その後で終了させるか続行させるか、いずれかを選ぶ必要がある。もしプロンプトが戻って来たら、`jobs` と入力する。おおよそ

```
$ jobs
[1]      Running                  emacs local-guide.tex
[2]+    Suspended                 a.out
```

のような出力が得られるはずである。悪さをしているのが、`a.out` なら、行頭の `[ ]` のなかの数字である `2` に `%` をつけた `%2` を用いて

```
$ kill %2
```

として、ジョブを止める。もし、正常に走っているが単に時間がかかっているだけだとわかっているのであれば、

```
$ bg %2
```

のように打ってバックグラウンドで実行してもよいだろう^{*18}。

さて、`ctrl-c` でも `ctrl-z` でもジョブが終了できないときには、別のターミナルを開き、(必要に応じて SSH ログインした後) `ps x` と打つ。すると、

```
$ ps x

  PID TT  STAT  TIME COMMAND
27515 p0  IW    0:03 -bash
27767 p0  TW    0:00 emacs
28123 p3  R     0:24 ./a.out
28529 p3  R     0:00 ps
```

^{*18} 最初からバックグラウンドで実行したいのであれば、コマンド実行時に最後に `&` をつければよい。例えば `a.out &` のように実行する。一方バックグラウンドで走っているジョブをフォアグラウンドに戻すには、`fg %2` などとする。

のように表示される。左からプロセスの ID 番号、制御端末、プロセスの状態を示す略号、現在までの CPU 時間、そして実行中のコマンドが表示されている。このなかで悪さをしているようなコマンドを探す。今の場合 `./a.out` が怪しいので、

```
$ kill 28123
```

として、そのプロセスを終了する。28123 は `./a.out` のプロセスの ID 番号である。ちゃんとプロンプトが帰ってくれば一件落着である。それでもだめなときは、

```
$ kill -HUP 28123
```

としてみる。まだだめな場合は `-HUP` を `-QUIT` ^{*19}にしてみよう。それでもうまくいかない場合は `-QUIT` を `-KILL` にするが、`-KILL` は最後の手段と考えて、無闇に使わないほうが無難である。

1.1.5 パイプとリダイレクション

UNIX の設計思想の一つに小さなツールを組み合わせるということがある。いまから紹介するのがその小さなツールを組み合わせるためのコマンドの書き方である。

パイプ

例えば、`ps` コマンドの出力の中から、`emacs` のプロセス ID を調べようとするとき、`ps` コマンドと文字列を探し出す `grep` コマンドを組み合わせ、

```
$ ps | grep emacs
```

のように実行する。“|” はパイプと呼ばれ、パイプの左側のコマンドが出力するデータをパイプの後ろのコマンドの入力に繋いでくれる。ここで使えるのは、画面に結果を書き出すコマンドとキーボードから入力を読み込むコマンド^{*20}の組み合わせだけであることに注意せよ。

パイプを使えば、`ls` などの出力で流れて行ってしまうものを、

```
$ ls -laF | more
```

のようにして、一時的なファイルを作る手間なしに、`more` を使ってゆっくり眺めることができる。

リダイレクション

次は、リダイレクションである。リダイレクションとは、通常画面に書かれる内容を、かわってファイルに書き出したり、通常キーボードから読まれる入力をファイルから読み込むようにする機能である。これを用いれば、コマンドの実行結果をファイルに保存したりすることも可能である。例えば、

```
$ ls -laF > ls.dat
```

のようにすれば、`ls -laF` の結果が `ls.dat` に書き出される。また、C のプログラムでの計算結果をいったんファイルに落とし、`Gnuplot` (1.4 節) で図にプロットするというといった作業も行うことができる。

^{*19} `-QUIT` を使うとプロセス (この場合は `a.out`) が実行中だったディレクトリに `core` というとても大きなファイルができることがある。通常は必要ないので消してしまってもかまわない。

^{*20} `grep` のマニュアルを見れば分かるが、`grep` は入力ファイルが指定されずに起動された場合は、標準入力 (ふつうはキーボード) を探すと書いてある。

1.1.6 コマンドヒストリとコマンドライン補完

コマンドヒストリとコマンドライン補完は、なれてしまうと手放せなくなる機能である。コマンドヒストリとは、一度実行したコマンドを呼び出す機能のことである。これを使えば、同じコマンドを実行するたびに最初から打つ手間が省ける。また、以前実行したコマンドをちょっと変えて実行するというのも簡単にできる。以前に実行したコマンドを呼び出すためには、プロンプトの所で `ctrl-p` を押すか、キーボードの上矢印を押す。プロンプトの後にコマンドが表示されるはずである。ここで `Enter` すれば、そのまま実行される。あるいは `ctrl-b` (左矢印) や `ctrl-f` (右矢印)^{*21} や `Backspace` あるいは `Delete` キーを使って編集してから実行することもできる。`ctrl-n` (下矢印) を押せば逆にヒストリを戻ってくることができる。また、

```
$ !string
```

とすれば、*string*^{*22} から始まる一番最近のコマンドを実行してくれる。さらに `ctrl-r` に続けて過去に打ったコマンドを頭から入力していくと、コマンドヒストリの中から逐次候補が表示される (逆インクリメンタルサーチ)。大昔に打ったコマンドを捜し出すのに便利である^{*23}。

```
$ history
```

と打つと、これまでに実行したコマンドの一覧が番号付きで表示される。“!`+ 番号`”で履歴中のコマンドを再度実行することができる。

```
$ !128
```

次は、コマンドライン補完である。例えば

```
$ ls -aF

./          excellent-bitmaps/  work/
../         save/
archive/    TeX/
emacs.doc   temp.out\(\ast\)
```

のような状況で次のようなコマンドを打ちたいとする。

```
$ chmod o-rx excellent-bitmaps
```

をいちいち最初から最後まで打っていたのでは、面倒である。そこでまず、

```
$ chm
```

まで打った所で、左の方にある `Tab` キーを押してみよう。すると、

^{*21} これらの矢印のついたキーのことをカーソルキーと呼ぶ。

^{*22} *string* は例である。もちろんどのような文字列でも構わない。

^{*23} 実際のところ、*!string* では、そのままコマンドが実行されてしまうので、`ctrl-r` の方が便利である。

```
$ chmod █
```

のように変わる。つまり、自動的にコマンドの一部を補完してくれるのである。さらにこの機能はコマンドだけではなく、ファイルやディレクトリの名前、ユーザー名にも使える。さっきの例では

```
$ chmod o-rx ex
```

まで打った時点で、`Tab`^{*24}を押せば、自動的に

```
$ chmod o-rx excellent-bitmaps
```

と補完される。これらの機能を利用すれば、これまで以上に効率よく UNIX とコミュニケーションを取れるはずである。

1.1.7 その他有用なコマンド

他によく使うコマンドを紹介しておこう。

- **who**
いま使っている計算機にだれがログインしているかを表示する。
- **clear**
ターミナル画面で、画面をクリアする。
- **cal**
今月のカレンダーを表示する。`cal 2024`とすれば 2024 年のカレンダーを表示する。
- **du**
あるディレクトリ以下について、ディレクトリごとにファイルの大きさの総和を取って表示する。`du -s .`として使用することで、カレントディレクトリ以下のファイルの大きさの総和を表示する。
- **tar**
ファイルをまとめて一つにまとめる。tape archiver の略である。ファイルを転送したり、バックアップを取るときに利用する。

```
$ tar cvf ../archive.tar .
```

のように使うと、`../archive.tar` というファイル^{*25}にカレントディレクトリ以下のすべてのファイルのバックアップが取られる。`.`の代わりに `*.c` を使えば カレントディレクトリの `.c` で終るファイルがまとめられる。また、`.`の部分にディレクトリを指定すれば、そのディレクトリ以下のファイルすべてのバックアップを取ることができる。反対に展開するときは、

```
$ tar xvf archive.tar
```

のようにする。

- **gzip**
ファイルを圧縮してサイズを小さくする。

^{*24} `Tab`でなく、`ctrl+d`を押せば、補完可能な候補の一覧が表示される。

^{*25} ファイル名の最後に `.tar` を付ける習慣にしておくと、あとで混乱が少なくなる。

```
$ gzip origin
```

のように実行すると、*origin* というファイルが消えて、*origin.gz* というファイル^{*26}が作成される。解凍^{*27}するには、

```
$ gzip -d oringin.gz
```

あるいは

```
$ gunzip origin.gz
```

のように実行する^{*28}。しばしば *tar* と組み合わせて使われる。

• *ln*

ファイルのリンク^{*29}を作成する。

```
$ ln filename linkname
```

のように実行する^{*30}と *filename* というファイルは *linkname* という名前でもアクセスできるようになる。これをリンク、とくにこの場合はハードリンクと呼ぶ。ハードリンクがコピーと違うのは、*filename* の内容を変更した場合、自動的に *linkname* の内容も変更される^{*31}ことと、ディスクの使用量がハードリンクを作っても (ほとんど) かわらないことである。また誤って *filename* か *linkname* のどちらかを消してしまっても一方は残っているので、バックアップの役にも立つ。しかし、ハードリンクは「自分のホームディレクトリ内」のように限られた空間^{*32}内でしか利用できない。

ハードリンクに対して、シンボリックリンクというものもある。シンボリックリンクは

```
$ ln -s filename linkname
```

のようにして作成する^{*33}。シンボリックリンクもハードリンクと同じく、*linkname* によって *filename* にアクセスすることが可能になる。しかし、*filename* を消したり、*filename* の名前を変更したりすると、もはや *linkname* でファイルにアクセスすることはできなくなってしまう。シンボリックリンクは、同一パーティション内にないファイルに対しても作成することができる。

• *time*

コマンドの実行時間を計測する。例えば

```
$ time gzip origin
```

^{*26} ファイルの最後に *.gz* が付く。逆にもし、ファイルの最後が *.gz* になっているファイルがあれば、それは *gzip* で圧縮されていると思ってよい。

^{*27} 圧縮を元に戻すことである。

^{*28} *tar* + *gzip* ファイル (ファイル名の末尾が *.tar.gz* になっているか、*.tgz* になっている場合が多い) を解凍・展開するときには、*gzip -cd archive.tar.gz | tar xvf -* のようにパイプをうまく使うと、中間ファイルの *archive.tar* を生成せずにいきなりディレクトリに展開してくれる。また、*tar zxvf archive.tar.gz* でも同じ結果となる。逆に、*tar zcvf ../archive.tar.gz .* で、圧縮されたアーカイブを直接作成することができる。

^{*29} コンパイラのところで登場する「リンク」という言葉とは関係ない。

^{*30} *cp* と同じ順序で引数を指定する。

^{*31} ハードリンクを作ると *filename* と *linkname* の区別はなくなり、両者は対等の関係になる。

^{*32} 同一パーティション内のこと。

^{*33} シンボリックリンクを作ること、を、「リンクを張る」といういいかたをすることがある。

のように、コマンドの前に `time` コマンドをつけて実行すると、コマンドの実行にかかった実時間 (real)、ユーザ CPU 時間 (user)、システム CPU 時間 (sys) が表示される。

1.2 リモートログインとファイル転送

UNIX では、SSH (secure **sh**ell の略) という仕組みを使うことで、ある計算機 (ログイン元、あるいはローカル計算機と呼ぶ) から別の計算機 (ログイン先、あるいはリモート計算機と呼ぶ) にインターネット経由でリモートログインして作業することができる。例えば、ECCS の端末 (iMac) から、物理学教室のワークステーション、あるいは地理的に離れた場所にあるスーパーコンピュータにログインして、あたかもその計算機が手元にあるかのように利用することができる。リモート計算機のホスト名やアカウントに関する情報は、あらかじめ、リモート計算機の管理者から入手しておく必要がある。

SSH では現在、2 種類の認証方式が使われている。ユーザ名とパスワードの組による認証と、公開鍵暗号による認証である。後者の方がよりセキュリティが高く、近年は公開鍵暗号のみに制限している計算機も多い。以下では、公開鍵暗号による SSH 接続について説明する。

1.2.1 公開鍵暗号方式

従来の暗号 (共通鍵暗号) では、暗号化と復号化に同一の鍵を用いるが、公開鍵暗号では、単一の共通鍵ではなく秘密鍵と公開鍵をペアを使う。この二つの鍵は、秘密鍵で暗号化したデータは公開鍵でのみ復号化でき、逆に公開鍵で暗号化されたデータは秘密鍵でのみ復号化できるという著しい性質を持つ。公開鍵暗号にもとづく SSH では、ローカル計算機であらかじめ秘密鍵と公開鍵のペアを作成し、そのうち公開鍵のみをログイン先のリモート計算機に登録しておく。実際の接続においては、まずリモート計算機側でユーザの公開鍵を使っているランダムな文字列が暗号化され、それが手元のローカル計算機に送られてくる。ローカル計算機で、その文字列をユーザの秘密鍵を使って復号してみせることで、そのユーザが登録された公開鍵の所有者であることを証明する。このようにして認証を行った上で、さらにその後の通信についても全て暗号化された上で行われる^{*34}。

重要な点は、リモート計算機に登録する公開鍵が仮に盗まれたり、他人に知られてしまっても、秘密鍵が知られない限り、なりすましや通信内容の傍受などの心配がないということである。一方、秘密鍵の方は絶対に人に知られてはならない。

1.2.2 鍵の作成と登録

秘密鍵と公開鍵の鍵ペアの作成は、ローカル計算機 (ECCS の iMac など) 上で、`ssh-keygen` コマンドにより行う。

```
$ ssh-keygen -t rsa
Generating public/private rsa key pair.
Enter file in which to save the key (/xxx/.ssh/id_rsa):(何も入力せず return)
Enter passphrase (empty for no passphrase):(パズフレーズを入力)
Enter same passphrase again:(パズフレーズを再度入力)
```

秘密鍵が `~/.ssh/id_rsa` に、公開鍵が `~/.ssh/id_rsa.pub` に作成される。ここで入力する「パズフレーズ」は、手元の計算機にログインするための「パスワード」とは別のものである。「パズフレーズ」は自分で決める文字列であり、「パスワード」とは異なるものでなければならない。また、設定した「パズフレーズ」は忘れずに覚えておく必要がある。

^{*34} 公開鍵暗号の仕組みは、暗号化だけでなく電子署名にも用いられる

作成された鍵のうち、公開鍵 (`id_rsa.pub`) をログイン先の計算機に登録する。公開鍵の登録は、リモート計算機の管理者の指示にしたがって行う。繰り返しになるが、秘密鍵 (`id_rsa`) は絶対に人に知られてはならない。公開鍵の代わりに間違えて秘密鍵を送ってしまわないよう注意が必要である。

`ssh-keygen` をオプションの指定なしで複数回実行してしまうと、既存の鍵が置き換えられてしまい公開鍵の登録からすべてやり直しになるので十分注意すること。

1.2.3 リモートログイン

リモート計算機へ SSH 接続するには、`ssh` コマンドを利用する^{*35}。例えば、ホスト名が `remote.phys.s.u-tokyo.ac.jp` という計算機に SSH ログインしたい場合、

```
$ ssh -X remote.phys.s.u-tokyo.ac.jp
```

と入力する。現在ログインしている計算機とログイン先のアカウント名が異なるときは、

```
$ ssh -X remote.phys.s.u-tokyo.ac.jp -l username
```

あるいは

```
$ ssh -X username@remote.phys.s.u-tokyo.ac.jp
```

のようにする。*username* はログイン先のアカウント名である。なお、`-X` オプションは、リモート計算機上で実行する Emacs、Gnuplot、Evince などのウィンドウをローカル計算機の画面で開くためのもの (X11 forwarding) である。

次に、パスフレーズを入力するように求められるので、鍵の作成時に設定したパスフレーズを入力すると、`remote.phys.s.u-tokyo.ac.jp` に接続され、プロンプトが表示される。ただし、初めて接続する計算機の場合、次のようなメッセージが出力される。

```
Host key not found from the list of known hosts.  
Are you sure you want to continue connecting (yes/no)?
```

ここで、`yes` と答えると、先に進むことができる。ログイン後、入力する命令は、すべてリモート計算機上で実行される。

作業終了後は、

```
$ exit
```

と入力すると、接続が解除され、ローカル計算機のプロンプトに戻る。

SSH 接続して作業していると、リモートとローカル、どちらの計算機の上で作業しているか分からなくなってしまうことがある。その際には、`hostname` コマンドを実行することで、作業している計算機のホスト名を確認することができる。

^{*35} 同様の機能を持つものに `telnet`、`rlogin` というコマンドがあるが、通信内容が平文でネットワーク上を流れるなど、セキュリティ上の問題があるので、今日では使われない。

1.2.4 多段 SSH

セキュリティの関係上、複数回の SSH を繰り返すことでしかログインできないこともある。この場合、途中経由する踏み台のリモートに対してローカルにある公開鍵を転送する必要がある。

例として 2 段 SSH の場合を示すと、

```
$ ssh -X -A username1@remote1.phys.s.u-tokyo.ac.jp
```

のようにして -A をつけて 1 段目の SSH を行い、更にそこから

```
$ ssh -X username2@remote2.phys.s.u-tokyo.ac.jp
```

のようにして 2 段目の SSH を行えばよい。最終段では -A がなくても構わない。

1.2.5 ファイルの転送

ある計算機から別の計算機にファイルをコピーしたい場合には、scp (secure copy の略) コマンドを用いる。ローカル計算機からリモート計算機 (例: remote.phys.s.u-tokyo.ac.jp) へファイル (report.pdf) を送る場合は、

```
$ scp report.pdf username@remote.phys.s.u-tokyo.ac.jp:~
```

とする。最後の~(チルダ) はリモート計算機のホームディレクトリ (/home/username など) を表す。コピー元のファイル名や、コピー先のディレクトリ名は適宜変更すること。また、.pdf という拡張子のつくファイルをすべて送りたい場合は、

```
$ scp *.pdf username@remote.phys.s.u-tokyo.ac.jp:~
```

とすればよい。report というディレクトリを送りたい場合は、

```
$ scp -r report username@remote.phys.s.u-tokyo.ac.jp:~
```

とする。逆に、リモート計算機のファイルをローカルへ取ってくる場合は、

```
$ scp username@remote.phys.s.u-tokyo.ac.jp:~/report.pdf .
```

などとする。最後の. (ドット) はカレントディレクトリを表す。scp コマンドは、ローカル計算機上で実行する必要があることに注意する。

ファイルの転送は、sftp (secure file transfer program の略) コマンドを用いて対話的に行うこともできる。

```
$ sftp username@remote.phys.s.u-tokyo.ac.jp
sftp>
```

sftp>は、sftp を実行中であることを示すプロンプトである。ここで、put コマンドを実行することで、ローカル計算機からリモート計算機へ、逆に、get コマンドでリモートからローカルへファイルをコピーすることができる。ls コマンドでリモート計算機のファイル一覧を表示、cd コマンドでリモート計算機上のディレク

トリ移動、`mkdir` コマンドでリモート計算機上のディレクトリ作成が行える。`lls`、`lcd`、`lmkdir` コマンドのように先頭に「`l`」を付けると、リモート計算機上ではなく、ローカル計算機上で一覧表示、ディレクトリ移動、ディレクトリ作成が行われる。リモート計算機とローカル計算機で今どのディレクトリにいるかを知るには、それぞれ `pwd`、`lpwd` コマンドを使う。

コピー作業終了後は、

```
sftp> exit
```

で `sftp` を終了する。

1.3 Emacs を使う

C 言語のプログラムや $\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}$ のソースコードなど、テキスト形式のファイルの編集には、エディタと呼ばれるソフトを用いる。UNIX で代表的なものとしては、Vim や Emacs がある。本節では Emacs の使い方を紹介しよう。

まず、Emacs を立ち上げるには

```
$ emacs
```

と入力すればよい。あるいは、

```
$ emacs &
```

としておけば、Emacs を別ウィンドウで立ち上げた後、元のターミナル内で別の作業を続けることができる。

1.3.1 チュートリアル

Emacs のコマンドの多くは `ctrl`-`なんとか` と `Esc`-`なんとか`^{*36}にキー定義されている。

Emacs のチュートリアル自体も、`ctrl`-`h` `T` に割り当てられている。`ctrl`-`h` `T` を押すときは、まず `ctrl`-`h` を押してコントロールキーを離してから `T` を押す。ここで `T` は大文字なので `Shift` キーと同時に押すことを忘れないこと。また、本来のコマンド名 `Esc`-`x` `help-with-tutorial` と打ってももちろんかまわない。以下、キーバインドの後にカッコ中に太字で書いてあるのがコマンドで、すべてのコマンドは `Esc`-`x` `コマンド名` と打つことでも、実行が可能である。このチュートリアルは基本的な事柄を網羅しているので、載っているキー定義を覚えてしまえば相当 Emacs が使いやすくなるはずである。

Emacs を使っていて何か困ったと思ったときは、まず `ctrl`-`g` (**keyboard-quit**) を押してみよう。それでもダメなら `ctrl`-`x` `ctrl`-`c` (**save-buffers-kill-emacs**) で Emacs から抜け出ればよい。

1.3.2 編集作業

それでは Emacs の中から、何かファイルを編集してみよう。Emacs が立ち上がった状態からファイルを編集し始めるには、`ctrl`-`x` `ctrl`-`f` (**find-file**) とする。

```
Find file: ~/
```

と聞いてくるので、既存のファイルを編集したいのならそのファイル名を、もし新たにファイルを作って編

^{*36} `Esc` は `Control` と違い、一旦 `Esc` を押して離した後に、他のキーを押す。

キー定義	コマンド	動作
	find-file	新たにバッファを作り、ファイルの名前を尋ねて読み込む
	insert-file	現在のカーソルの位置に指定したファイルを挿入する
	save-buffers-kill-emacs	Emacs を終了する。未保存のバッファは保存するかどうか尋ねてくる
	save-buffer	バッファの内容をファイルに書き出す
	write-file	バッファの内容を書き出すファイルの名前を尋ねてから書き出す
	keyboard-quit	コマンドの入力を中断する
	scroll-up	画面 1 枚分上にスクロールする
	scroll-down	画面 1 枚分下にスクロールする
	isearch-forward	下向きに文字列を検索する。検索モードから抜けるには、 を打つ
	isearch-backward	上向きに文字列を検索する。検索モードから抜けるには、 を打つ
	query-replace	文字列の置換を行う。スペースで実行、“n” で不実行 ! で合致するすべてを置換する
	kill-line	現在いる行のカーソルより後ろを削除する
	set-mark-command	マークを設定する
	kill-region	マークから現在のカーソルの位置までを削除する
	yank	カーソルの位置に削除した内容を戻す
	advertised-undo	アンドゥ、直前の操作を取り消す
	beginning-of-buffer	文章の始めに飛ぶ
	end-of-buffer	文章の終りに飛ぶ
	split-window-vertically	ウインドウを二つに分割する
	delete-other-windows	今カーソルのいるウインドウを残し、他のウインドウを消す
	other-window	ウインドウが分割されているとき、違うウインドウに飛ぶ
	make-frame	新たにフレームを作る
	delete-frame	現在のフレームを消す
	switch-to-buffer	他のバッファにスイッチする
	list-buffers	現在のバッファ一覧を出す
	recenter	ウインドウをリフレッシュし、カーソル行をウインドウ中央に移動する
	toggle-egg-mode	かな変換モードを on/off する
	start-kbd-macro	キーボードマクロの定義を開始する
	end-kbd-macro	キーボードマクロの定義を終了する
	call-last-kbd-macro	一番最近定義したキーボードマクロを実行する

表 1.1 Emacs の主なコマンド

集を始めたいのならそのファイル名を入力する*37。ここでは、ために trial.c というファイルを編集してみよう。

普通のキーは打てばそのままキートップに書いてある文字が挿入される。間違えた時には あるいは キーで消す。カーソルキーを使ってカーソルを移動することも可能である。カーソルキーを使わずに、 (**previous-line**)、 (**next-line**)、 (**forward-char**)、 (**backward-char**) でも構わない。

Emacs はファイルの名前から判断して、これから編集するのが C のプログラムだと認識する。Emacs のウインドウの下にあるモードライン (反転表示されている 1 行のこと) に C の文字が見えるはずである。

C のプログラムは、通常次のようにブロックをインデントする。

```
int main(void) {
    return 0;
```

*37 ここで読み込まれたファイルはバッファと呼ばれる一時的に確保される記憶領域に置かれる。編集の操作はすべて、バッファに対して行われるので、ファイルに書き出さないと編集の結果は残らない。

```
}
```

インデントを行うのに、いちいちその数だけスペースを打つのは面倒である。Emacs は C のファイルを編集していると認識すると、`Tab` を打つことで適当な場所までインデントを行ってくれる。

何か文章やプログラムを Emacs 上で書いたとしよう。編集したファイルを保存するには、`ctrl-x ctrl-s` (`save-buffer`) を使う。

```
Wrote /home/s001500/trial.c
```

のような表示が Emacs のミニバッファに出るはずである。Emacs の隣のターミナル画面で `ls` コマンドを実行して `trial.c` というファイルが新たに生成されていることを確かめよう。さらに、`cat` とか `more` のようなコマンドでファイルの中身を確認してみるのもよい。

さて、Emacs での作業も終わったので、Emacs から抜け出すことにしよう。Emacs から抜けるには、`ctrl-x ctrl-c` (`save-buffers-kill-emacs`) とする。

1.3.3 Emacs の様々なコマンド

Emacs には実に様々なコマンドがあるが、その中でも使用頻度の高いコマンドを表 1.1 にまとめておく。コマンドは、`Esc-x` コマンドと打つことでも実行できる。

ある領域を削除、移動、コピーしたいときには領域の先端でマークし、領域の終端までカーソルを移動した後、`ctrl-w` を押して領域を削除する。移動したいときには、さらに移動先までカーソルを持って行き、そこで `ctrl-y` を押す。コピーしたいときには、削除した直後に `ctrl-y` を押して復旧し、さらにコピー先までカーソルを持って行って、`ctrl-y` を押してコピーする。つまり一旦 `ctrl-w` でためておいてから^{*38}、`ctrl-y` であちこちにコピーする。また、`ctrl-k` を連続して使って消した場合も、`ctrl-y` でまとめてペーストすることができる。そのほか便利な機能としては、`Esc-x goto-line` で指定行に飛ぶなどが挙げられる。

1.4 Gnuplot を使う

計算結果をグラフに変換するツールはいろいろあるが、UNIX で最もよく使われているものの一つに Gnuplot がある。この節では、Gnuplot の基本的な使い方を解説する。

1.4.1 Gnuplot の起動とデータのプロット

ターミナルで

```
$ gnuplot
```

と入力すると、Gnuplot が起動し、プロンプトが表示される。

```
gnuplot>
```

終了するには、

```
gnuplot> exit
```

^{*38} 実は `Esc-w` だと削除しないでためるだけなので、こちらの方が楽である。

と入力する。また、

```
gnuplot> help
```

あるいは、

```
gnuplot> help command
```

でコマンドの説明が表示される。

次に、データをプロットしてみよう。Emacs や Vim などのエディタを使って、data.dat という名前で、中身が

```
1.0 2.0
2.0 4.0
3.0 6.0
4.0 8.0
5.0 10.0
```

のようなファイルを用意し、

```
gnuplot> plot 'data.dat'
```

と入力するとプロットされる。データファイルには1行に「横軸の値と縦軸の値」を入れる。グラフの右上にデータセット名(デフォルトではファイル名)を示す「data.dat」が表示される。これは任意の名前あるいは非表示に変更できる。

上の例では、データファイル data.dat が置いてあるディレクトリで Gnuplot を起動している。別ディレクトリにあるデータファイルをプロットするには、絶対パスか相対パスでファイル名を指定するか、

```
gnuplot> cd 'directory'
```

でデータファイルの置いてあるディレクトリに移動した後、プロットすればよい。現在どのディレクトリにいるかを知るには `pwd` コマンド (`print working directory` の略) を使う。

```
gnuplot> pwd
/Users/...
```

1.4.2 スクリプトファイルの読み込み、書き出し

Gnuplot は1行ずつコマンドを実行しプロットしていく対話型スタイルが基本であるが、複雑なプロットの場合には、実行したいコマンドをスクリプトファイルの形であらかじめ用意しておき、一括して読み込む方が便利である。例えば、Emacs や Vim などのエディタを使って、plot.plt という名前で、中身が

```
plot 'data.dat'
```

のようなファイルを用意し、Gnuplot の中で

```
gnuplot> load 'plot.plt'
```

と入力すると、上の例と同じプロットが表示される。あるいは、ターミナルで直接

```
$ gnuplot plot.plt
```

としてもよい。環境によっては、プロットが一瞬だけ表示されてすぐに消えてしまうが、その場合は、

```
$ gnuplot -persist plot.plt
```

とすればよい。

対話形式で作成したプロットをスクリプトファイルに保存することもできる。

```
gnuplot> save 'plot2.plt'
```

以下で説明する x 軸や y 軸、キャプション、矢印の設定など、その時点での全ての設定が保存される。保存したスクリプトファイルは `load` コマンドで再度読み込むことができる。

1.4.3 プロットスタイル

関数をプロットするには、

```
gnuplot> plot sin(x)
```

のようにすればよい。次のように関数を定義して使うこともできる。

```
gnuplot> f(x) = sin(x)  
gnuplot> g(x) = A*cos(x)*exp(x)  
gnuplot> A = 10.0  
gnuplot> plot f(x), g(x)
```

関数の値のサンプリングのピッチを変えるには、

```
gnuplot> set sample 10000
```

とする。

一方、データファイル内のデータをプロットするには、引用符内にデータファイル名を指定する。

```
gnuplot> plot 'filename'
```

`plot` コマンドには様々なオプションが指定できる。以下、代表的なものを挙げる。

- `index 0:0`
データファイルの内、最初のデータセットを使う (省略可)
- `using 1:3` (`u 1:3` と省略可。 `using ($1):($3)` と等価)
データセットの 1 列目を x 座標、3 列目を y 座標としてプロットする
- `using 1:($2*10)`
データセットの 2 列目の値を 10 倍したものを y 座標としてプロットする

- `using 1:($2+f($1))`
関数 $f(x)$ を定義しておき、その関数値をプロットすることもできる
- `with line` (`w 1` と省略可)
データを線で結ぶ
- `with linespoints`
データをシンボルで表示し、さらに線で結ぶ
- `using 1:2:3 with yerrorbars`
 y 座標の値にエラーバーを付ける。エラーバーとしてどの列を使うかは `using` オプションで指定する
- `using 1:2:3 with yerrorlines`
 y 座標の値にエラーバーを付け、線で結ぶ
- `title 'hoge'` (`ti 'hoge'` と省略可)
図の凡例の文字列を指定する
- `notitle`
凡例をつけない
- `linewidth 3` (`lw 3` と省略可)
線の幅を 3 倍にする (論文用の図としては `lw 3` くらいがよい)
- `linecolor 7` (`lc 7` と省略可)
線の色を指定する。青は 6、赤は 7 など。
- `dashtype 2` (`dt 2` と省略可)
2 から 5 まで 4 種類の破線や点線が選べる
- `pointtype 2` (`pt 2` と省略可)
データ点のシンボルを指定する。1 は十字、2 は X 印、4 は四角など

これらを応用すると、次のようなことができる。

```
gnuplot> plot 'data.dat' with line linecolor 6, 'data2.dat' linecolor 7
```

では、`data.dat` の点は青線でつながり、`data2.dat` は赤点のみでプロットされる (`data2.dat` も自分で用意すること)。また、点ではなく常につながった線で表示させるには、

```
gnuplot> set style data line
gnuplot> plot 'data.dat', 'data2.dat'
```

とする。

直前のプロットを再プロットするには、

```
gnuplot> replot
```

とする。

1.4.4 3次元プロット

2次元関数をプロットするには `splot` コマンドを使う。

```
gnuplot> splot x*y
```

メッシュの幅を指定するには、

```
gnuplot> set isosamples 20
```

とする。

データファイルから 3 次元プロットするには

```
gnuplot> splot 'filename' using 1:2:5
```

のようにする。using オプションで、 x 座標、 y 座標、関数値の列を指定する。2 次元プロットと同様に、エラーバーをつけることも可能である。

1.4.5 x 軸・ y 軸の設定

x 軸を log スケールにするには、

```
gnuplot> set log x
```

x 軸をリニアスケールにする (戻す) には、

```
gnuplot> unset log x
```

y 軸を log スケールにするためには、

```
gnuplot> set log y
```

などとする。

x 軸にラベルを付けるには、

```
gnuplot> set xlabel 'x axis'
```

x 軸のプロット範囲を指定するには、

```
gnuplot> set xrange [-10:10]
```

のようにする。 x 軸上に長めの目盛り (tic) を 5 おきに、数値とともに打つには、

```
gnuplot> set xtics 5
```

とする。あるいは、 x 軸の目盛りを -3 から 1 おきに 5 まで振るには、

```
gnuplot> set xtics -3,1,5
```

とすればよい。 x 軸の長めの目盛りの間を 10 分割して、短い目盛りを振るには、

```
gnuplot> set mxtics 10
```

とする。 y 軸に関しても同様である。

プロット中に $y = 0$ の軸を引くには、

```
gnuplot> set xzeroaxis
```

とする。

1.4.6 表題・凡例・図の形

グラフそのものに表題を付けるには、

```
gnuplot> set title 'Title of this plot'
```

とする。レジェンド (凡例) の表示位置は、標準で右上 (right top) に表示されるが、位置を変更するには

```
gnuplot> set key left bottom
```

のようにする。レジェンド全体を非表示にするには、

```
gnuplot> unset key
```

とする。

図全体の形は `set size` コマンドで変更できる。図を真四角にするには、

```
gnuplot> set size square
```

図の縦横比を 1:1.5 にするには、

```
gnuplot> set size ratio 1.5
```

とする。

1.4.7 矢印・文字列

図中に矢印や文字列を表示することができる。 (x_1, y_1) から (x_2, y_2) に矢印を引くには、

```
gnuplot> set arrow from x1,y1 to x2,y2
```

とする。また、位置 (x, y) に文字列を表示するには、

```
gnuplot> set label 'text' at x1,y1
```

とすればよい。

1.4.8 文字のスタイル

表題や凡例、軸のラベルなどに使う文字のサイズやフォントは変更することができる。また、ギリシャ文字や上付き・下付き文字を使うこともできる。以下に例を示す。

- `'{/=36 energy}'`
36 ポイントの大きさで表示される

- `'{/=36 {/Arial-Italic E}}'`
36 ポイント斜体で E のように表示される
- `{/=36 {/Symbol D}}'`
ギリシャ文字の Δ
- `'{/=36 {/Symbol c}^{\mathrm{total}}_e}'`
 χ_e^{total}

1.4.9 PDF ファイルへの出力

図を PDF 形式でファイルに出力するには、次のようにする。

```
gnuplot> set terminal pdfcairo color enhanced
gnuplot> set output 'figure.pdf'
gnuplot> load 'figure.plt'
gnuplot> set output
gnuplot> set terminal pop
```

プロットの内容が PDF 形式で figure.pdf に出力される。出力された図は、 $\text{\LaTeX}$ に取り込むことができる。なお、最後の 2 行は元の設定 (画面上での表示) に戻すためのコマンドである。これを実行しておかないと、以降の操作で figure.pdf が上書きされてしまうので注意が必要である。

第2章

C 言語入門

プログラミング言語とは、コンピューターに仕事をさせるために、その作業内容を指示するための特別な言語である。この言語で書かれた作業の手順書を「プログラム」とよび、プログラムを作成することを「プログラミング」という。プログラミング言語には、さまざまな種類がある。C 言語はもともと、UNIX (マルチユーザ、マルチタスクのオペレーティングシステム) を記述するためのシステム言語として開発された。現在は、システムソフトウェアの作成だけでなく、事務処理や科学技術計算、アプリケーションソフトウェア (表計算やワープロなど) の開発など、広く汎用プログラミング言語として使用されている。

2.1 C 言語の基礎知識

2.1.1 なぜ C 言語?

Fortran や C くらいしかなかった時代と違い、最近では様々な (多くの場合 C よりもかなり書きやすい) プログラミング言語がある。そんな中どうしてわざわざ手間のかかる C 言語を学ぶ必要があるのだろうか?

■処理速度が速い

多くの場合莫大な計算を行う必要のある科学技術計算において、処理速度が遅いというのは致命的である。C は数あるプログラミング言語の中でも最速の部類に入り^{*1}基礎的な処理能力が非常に高いので、こういった用途に向く。

■機能が必要最低限で把握しやすい

C の言語機能は非常に限定的であり (もちろんそれに起因する苦勞もあるのだが) 全体を比較的簡単に眺めることができる。エラーログも簡素で読みやすい。

■様々な環境で利用できる

PC だけではなく、ワークステーションやスーパーコンピュータ上でも利用可能である。環境依存性も少なく安心して利用できる。また、実験装置・解析装置でも使われる FPGA や組み込みコンピュータ環境においても利用できる。メモリの管理についてもユーザからよく見えるので、メモリ量が限られた環境での利用にも適している。

■素直に実装できる

低速な高級言語^{*2}では少々無理矢理にでもうまく計算が重い部分を外部ライブラリ (多くの場合 C で書かれている) に押し付ける工夫が必要になる。また、高速な高級言語^{*3}であってもバックエンドとして走るコンパイラにとって分かりやすい書き方をしなければ急速にパフォーマンスが落ちる。つまり、高効率なプログラムを書きたいときには“ただ考えた内容をそのままプログラムにする”だけではうまく行かないのである。対照的

^{*1} 太刀打ちできる言語は C++・Rust・Fortran くらいしかない。

^{*2} Python など。

^{*3} Julia など。

に、C は基本的にあらゆる操作が充分高速であるのでこういった特殊な気遣いをする必要はあまりない。ただやりたいことをそのままプログラムに焼き直せば大抵の場合うまくいく。

2.1.2 まずはコンパイル

ここでは、最も短い“コンパイル可能な C のプログラム”を作成し、それをコンパイルしてみよう。

■コンパイルとは？

C, C++, Fortran などの言語は「コンパイル言語」と呼ばれる。これらの言語は、処理内容を記述したファイル（いわゆるソースコード）を直接実行することはせず、一度コンパイラというソフトウェアにソースコードを処理させ、出力された実行可能なファイルを実行対象とする。この操作をコンパイルという。

一方、コンパイルして計算機の言葉にするのではなく、ソフトウェアがプログラムを直接理解してその内容を実行するプログラム言語（スクリプト言語）もある。この「プログラムを直接理解するソフトウェア」のことをインタプリタといい、「プログラム」のことをスクリプトという。sh、Javascript、Ruby、Python、Julia などが代表的なインタプリタである。

性能が要求されるような状況下ではコンパイル言語を使用するのがよいが、事前にコンパイルしておかなくてはならない都合上あまり柔軟な操作はできない。対してスクリプト言語は実行時まで曖昧を残しておくことやインタラクティブな編集が出来る反面、動作速度に関してはコンパイル言語に及ぶべくもない。用途に応じて適切に使い分けよう。

■型の概念について

C 言語の学習において最初の壁となるのは型の概念であろう。高性能なプログラムを書く場合には変数の素性のある程度束縛する型の概念が不可欠となる。

主な変数の型としては以下のようなものがある。

int	整数型
unsigned	符号なし整数型
float	単精度浮動小数点数型
double	倍精度浮動小数点数型
char	文字型

“精度”は変数 1 つを記憶するのにどれだけのリソースを使うかに深く関係しており、一般にたくさんの bit を使う型のほうが精度や表現能力が高い。よって例えば double でもよい場合はわざわざ精度が低い float を使う利点はない。

各型の変数で表現できる値の範囲については状況依存であるが、典型的に以下のようにになっている*4。

int	$-2^{31} \sim 2^{31} - 1$
unsigned	$0 \sim 2^{32} - 1$
float	$0.0 \text{ と } \pm(1.40129846432481707 \times 10^{-45} \sim 3.40282346638528860 \times 10^{+38})$
double	$0.0 \text{ と } \pm(4.94065645841246544 \times 10^{-324} \sim 1.79769313486231570 \times 10^{+308})$

2024 年現在、int は 32bit が主流となっている。

■プログラムの構成

C のプログラムの最小単位は関数である。“C のプログラムは関数を並べたものである”ということもできる。関数を定義するときは以下のように書く。

型 関数名 (引数) { 処理 }

*4 整数型は 1 刻みで誤差がないのに対し、浮動小数点数型は値が大きくなるほど隣接する数字との間隔が増大して精度が低下することに注意せよ。

関数名

関数の名前は自由につけられる。ここで、C でプログラム中に使う名前 (関数以外でも共通) には

- 先頭は英字
- 2 文字以降は英数字または `_` (下線)
- `_` (下線) を 2 つ以上連続させてはならない
- 大文字/小文字は区別される
- 名前に重複があってはならない

という制約がある。

引数

関数には引数 (パラメタ) を渡すことができるが、その型はあらかじめ指定しておかねばならない (具体的な例については 2.7 節参照)。渡さない場合には `void` と書く。

型

関数は何かを処理して最後に値を返す (関数値) もののことである。C 言語の関数はあらかじめ決められた型の値しか返すことができない。

処理

この中で、1) 関数が行う処理内容を書き、2) 最後に関数値を返す。関数値を返すには

```
return 関数値;
```

と書く。最終的にどこかに到達することさえ保証されていれば `return` が複数個あっても構わない。

プログラム中に関数は複数作ることができるが、必ず `main` という `int` を返す関数が必要である。この事情については後ほど説明することとして、ひとまず C における関数の例を示す。

例 2.1.1.

```
int main(void) { return 0; }
```

■コンパイル

Emacs などのテキストエディタを使って例 2.1.1. の 1 行を入力したテキストファイルを作成し、`2.1.1.c` という名前で保存する。このプログラムをコンパイルするには C 言語用のコンパイラ `gcc` を用いる。

```
$ gcc 2.1.1.c -Wall -Wextra
```

`-Wall -Wextra` というのは警告メッセージを最大限出力させるためのオプションであり、必ずつけたほうがよい。同一ディレクトリに `a.out` というファイルができていたので、以下のように実行する^{*5}。

```
$ ./a.out
```

(ただし、現段階のプログラムでは、実行しても何も起こらない。)

■main について

C 言語において必ず必要になる `main` は“そのプログラムがどんな処理をするか”を記述した関数である。つまり、プログラムを実行した際に実行される部分は `main` の中身だけであり、他の部分に書いたものは `main` から何らかの形でアクセスしない限り完全に無視される。`main` が `int` 型でなくてはならないのは歴史的経緯に

^{*5} . がカレントディレクトリを表していたことを思い出そう。

よる*⁶。

2.1.3 書式の慣例

先に進む前に、C のプログラムの書式の慣例について、いくつか説明する。

■自由書式 (free format)

C 言語では基本的に書式に制限がない (自由書式)。文の終わりは、記号 ; で判別する。一つの行に二つの文を書くこともできるし、

```
aaaaa; bbbbbbb;
```

一つの文を二つの行にわけて書いても問題ない。

```
aaaaaaaaa
aaaaaaaaa;
```

例えば、例 2.1.1. は、以下のようにも書くことができる。

例 2.1.2.

```
int main(void) {
    return 0;
}
```

本書では、例 2.1.2. のような書式を用いることにする。

■インデントの慣例

C の文は、意味上のレベル (入れ子構造の階層) を持つ。意味上の各レベルをわかりやすくするため、より深いレベルを右側に段付けして書くことが多い (インデント)。

```
aaaaa      /* ← 最上位のレベルの文 */
aaaaa
    bbbbb   /* ← 一つ深いレベルの文 */
    bbbbb
        cccc /* ← もう一つ深いレベルの文 */
        cccc
            bbbbb
aaaaa
```

例 2.1.2. の関数は、すでにインデントして書かれている。インデントは上の行から行毎に順にキーボード上の **Tab** を押せばできる。

■コメント

/* と */ で囲った部分はコメントとみなされる (複数行も可)。1 行だけコメントアウトしたいときには//が便利。

例えば、例 2.1.2. にコメントを挿入してみると、

例 2.1.3.

```
int main(void) {
```

^{*6} main が返した値は外部から終了コードとして参照できるようになっていて、main が正常に終了したかを確認することができる。正常終了したときには 0 を、異常終了したときにはそれ以外を返すのが一般的。

```

    /* これはコメント。
    複数行にわたっても OK。 */
    return 0;
}

```

のようになり、`/*` と `*/` の間にある部分はコンパイルするときに無視される。

■自動整形について

プログラムの可読性を担保する上で、インデントやスペースの入れ方などの記法を統一することは極めて重要である。これを全て自分の手で行うのは非常に手間がかかるので、便利な自動整形ツールを使うとよい。C 言語用としては clang-format が有名である^{*7}。この冊子にあるプログラムはすべて clang-format で整形されている。

2.1.4 コンパイル (2)

次に、“何かがおこる”プログラムをコンパイルしてみよう。

例 2.1.4.

```

#include <stdio.h>

int main(void) {
    printf("Hello, C!\n");
    return 0;
}

```

例 2.1.4. のプログラムを Emacs などで作成し、2.1.4.c という名前で保存する。ここで、`printf` は、文字を出力する標準ライブラリ関数である (詳細については後ほど説明する)。この例は “Hello, C!” を出力するもので、`\n` は改行を表す特殊文字である。

それでは、保存したプログラムを、`gcc` を用いてコンパイルし、実行してみよう。

```

$ gcc 2.1.4.c -Wall -Wextra
$ ./a.out
Hello, C!

```

コンパイル時にエラーで止まってしまう場合には、ソースコードをもう一度見直すこと。

また、次のようにすれば、`a.out` ではなく好きな名前の実行ファイルが作成できる。名前は `.out` で終わるものでなくてもよい。

```

$ gcc -o hello 2.1.4.c -Wall -Wextra

```

この場合は同一ディレクトリに `hello` というファイルができ、以下のようにすれば実行できる。

```

$ ./hello
Hello, C!

```

■新しく出てきた概念について

^{*7} 例えば Python では `autopep8` や `black`、`LATEX` では `latexindent` が使用できる。

```
#include <file>
```

file の中身をこの場所に展開する。ファイルを探しに行く場所はあらかじめ決められており、そこに同名のファイルがなければエラーとなる。

```
stdio.h
```

標準ライブラリ関数 `printf` 関数を使う際に必要なファイル。システムの奥深くにある特別な場所に配置されており、特になんの対策も講じることなく使用できる。

2.1.5 ライブラリのリンク

プログラムの中で $\sin x$ や $\cos x$ などの数学関数を用いた場合は、前述の方法ではコンパイルに失敗する^{*8}。まずは、例 2.1.5. のプログラムを作成し、2.1.5.c という名前で保存しよう。

例 2.1.5.

```
#include <math.h>
#include <stdio.h>

int main(void) {
    const double angle = 60.0 * M_PI / 180.0;
    printf("cos(%lf) is %lf\n", angle, cos(angle));
    return 0;
}
```

このプログラムでは `math.h` 内で宣言されている数学関数を使っているため、`libm` というライブラリが必要となる。ライブラリとは何かについてはさておくこととして、以下のようにすることでコンパイルができる。

```
$ gcc 2.1.5.c -lm -Wall -Wextra
```

`-lm` という部分は、`m` (`libm` のうち `lib` を削除した残り) を `-l` というオプション引数を使って `gcc` に渡す構文である。例えば、`libsocket` を使いたい場合は、`-lsocket` と指定する。`math.h` の中には `sin` や `cos` だけでなく、例 2.1.5. で使われている π ($=3.1415\dots$) も `M_PI` として定義されている。定数 `M_PI` は C の標準機能ではないため、`gcc` 以外のコンパイラを使用したり、`gcc` にオプションをつけて使用するとエラーとなることがあるので注意せよ。

■ライブラリについて

ライブラリとは、大雑把に言うところ“既にコンパイルがほぼ済んだ状態で準備された関数”を呼び出すために必要なファイルである。今回使用した `libm` は数学関数をまとめて定義したもので、`math.h` とはまた別の特別な場所に格納されており、`-l` オプションでリンクするだけで呼び出すことができるようになる。

■新しく出てきた概念について

`math.h`

数学関数をまとめて定義したヘッダ。使用するには `libm` が必要。

算術演算子

以下の 5 種類がある。

^{*8} 数学関数の場合、macOS ではエラーにならない。しかし、一般的なライブラリの中で定義されている関数を使う場合には適切なリンクが必要である。

加算	+
減算	-
乗算	*
除算	/
剰余	%

整数を整数で割るとあまりを切り捨てる整数の割り算になることと^{*9}、冪乗演算子は存在しないことに注意せよ。

変数定義

`const double angle = 60.0 * M_PI / 180.0;` の部分で `angle` という変数を定義して代入している。今回は先頭に `const` をつけたのでこれ以降この変数を変更することはできない^{*10}。

■printf について

例 2.1.5. だけでなく例 2.1.4. でも出てきた、`printf` という関数 (**print with formatting** の略) は画面に文字や数字を表示させる場合に用いられ、

```
printf("フォーマット", 変数 1, 変数 2, .....)
```

という形で使用する。例 2.1.5. の場合はフォーマットが “`cos(%lf) is %lf\n`” になっているので、前者の `%lf` の部分が変数 1 (`angle`) の値に置き換えられ、後者の `%lf` の部分が変数 2 (`cos(angle)`) の値に置き換えられて表示される。`%lf` の他に `%d`, `%s`, `%f` などがあり、変数の型によって使い分ける必要がある。

<code>%d</code>	<code>int</code>
<code>%u</code>	<code>unsigned</code>
<code>%f</code>	<code>float</code>
<code>%lf</code>	<code>double</code>
<code>%c</code>	<code>char</code> 文字
<code>%s</code>	<code>char*</code> 文字列

`%` の部分では型だけでなく書式の指定もできる。例えば `%.20lf` とすると小数点以下 20 桁が表示されるようになる。

ここから先では他にも様々な標準ライブラリ関数を使用するが、その詳細については基本的に省略する。`man` や Web 検索で調べてほしい。

2.2 制御文

2.2.1 C における論理値

古い規格の C には真 (`true`) と偽 (`false`) の 2 値しかとらない論理型 (いわゆる `bool`) がない^{*11}。そこで真の代わりに `≠ 0` (典型的に 1 を用いるが、それ以外でも構わない)、偽の代わりに 0 を用いる。

以下の比較演算子・関係演算子を使用できる。

^{*9} 例えば、`3.0/2` は 1.5 となるが、`3/2` は 1 になる。

^{*10} 変更する予定のない変数には積極的に `const` をつけるようにしよう。これだけでかなりデバッグがしやすくなる。

^{*11} 新しい規格では一応存在することになっているが、言語機能として備わっているとは言い難い。

a と b が等しい	a == b
a と b は等しくない	a != b
a は b より大きい	a > b
a は b より小さい	a < b
a は b 以上	a >= b
a は b 以下	a <= b

浮動小数点に対して==を用いると誤差無しの完全一致を要求すること (数値誤差があるのでほとんど確実に偽である) に注意せよ。

また、論理演算子には以下のようなものがある。

a または b	a b
a かつ b	a && b
a でない	!a

2.2.2 ブロックについて

{ } で囲われた部分をブロックという^{*12}。新しい規格の C 言語では変数の宣言をあらゆる場所に書くことができる。あるブロックの中で定義された変数はそのブロックの中でしか使えないが、逆に外にある変数をブロックの中で参照したり、変更することはできる。この性質をうまく用いることで、プログラム中のごく限られた領域でしか使用しないはずの変数を誤って他の場所で使ってしまうという厄介なバグをかなり抑制することができる。

2.2.3 if 文

「... ならば... を実行して、それ以外なら... を実行する」という内容のプログラムは if ~ else ~ を用いて書く。

例 2.2.1.

```
#include <stdio.h>

int main(void) {
    const int a = 10;
    const int b = 20;
    if (a == b) {
        printf("a is equal to b\n");
    } else {
        printf("a is not equal to b\n");
    }
    return 0;
}
```

この例では、a と b が異なるので、実行すると a is not equal to b と表示される。

if 文は一般に以下のような形をとる。

```
if (A) {
    ブロック A
} else if (B) {
    ブロック B
}
```

^{*12} 既に関数定義の書式として出てきた。


```

} else if ... {
    ...
} else {
    ブロック C
}

```

処理が if に到達すると上から順番に () の中身がチェックされていき、一番最初に条件を満たしたブロックの中を実行してそれ以降は無視する (else まで来たらブロック C を実行する)。else if はいくつあってもよいし、else はなくても構わない。

単純な if をより簡潔に書くための構文として三項演算子 (?:) というものがある。調べてみよ。

2.2.4 for 文

繰り返しを行うためには、for を用いる。次の例は 1 から 100 までの和を計算するものである。

例 2.2.2.

```

#include <stdio.h>

int main(void) {
    int sum = 0;
    for (int i = 1; i <= 100; ++i) {
        sum += i;
    }
    printf("sum of integers from 1 to 100 is %d\n", sum);
    return 0;
}

```

for 文は以下のような形になる。

```

for (A; B; C) {
    ブロック
}

```

処理が for に到達するとまず A が実行され、それ以降は“B を評価 → B が真ならブロックの中身を実行し、偽なら for から抜ける → C を実行”の組み合わせを繰り返す。A,B,C は空欄でも構わない。

例 2.2.2. における int i のような、for ループのカウンターとして使用する変数は A の部分で定義するのが好ましい^{*13}。

■新しく出てきた概念について

++

++i は $i = i + 1$ と同じ意味を持っている。この ++ をインクリメント演算子という。同様に、 $i = i - 1$ と同じ意味を持つものとして --i がある。これをデクリメント演算子という。この他にも、i++ と i-- という書き方もある。これらの違いは、インクリメントされたりデクリメントされたりするタイミングが異なることにある。もし、

```

int i = 10;
int j = i++;

```

ならば、j の値は 11 ではなく 10 となる (j に代入した後で i を 1 増やす)。ところが、

^{*13} ここで定義した変数は for のブロック中でのみ有効なので、他の場所で誤って参照したり名前が衝突する心配がなくなる。

```
int i = 10;
int j = ++i;
```

ならば、j の値は 10 ではなく 11 となる (i を 1 増やしてから j に代入する)。特別な理由がない限り、++i と --i を使うのがよい^{*14}。

複合代入

今回使用した a += b は a = a + b と同じ意味の式である。+= を使ったほうが短く書ける上、場合によっては前者のほうが高速になる場合もある^{*15}ので、可能な限りこちらを使うほうがよい。

全く同様の記法が加減乗除と剰余全てについて使用できる。

2.2.5 while 文

ある条件が満たされている間、何かを繰り返し実行したいときは、while を使う。次の例は、i が 0 より大きい間は while ブロックを実行し、その結果として、1 から 100 までの和を求めるものである^{*16}。

例 2.2.3.

```
#include <stdio.h>

int main(void) {
    int i = 100;
    int sum = 0;
    while (i > 0) {
        sum += i;
        --i;
    }
    printf("sum of integers from 100 to 1 is %d\n", sum);
    return 0;
}
```

while 文は次のように動作する。

```
while (A)
{
    ブロック
}
```

while に処理が到達すると、“A を評価 → A が真ならブロックを実行”の組み合わせを繰り返す (初回チェックで A が偽ならブロックの中身は一度も実行されない)。while(1) と書くと A が常に正しいとして扱われるので、次に説明する break を呼ぶまで何度でもループし続ける。

2.2.6 break 文

for や while のような繰り返しを行う文の中から途中で抜きたい場合には、break を用いる。例 2.2.4. では、cos の値が負になったとき強制的に for 文を抜けている。

例 2.2.4.

```
#include <math.h>
#include <stdio.h>
```

^{*14} i++ は一度もとの i の値を別の場所に保存しておく必要があるので少しだけ計算量が多い。

^{*15} 前者は a を直接置き換えてよいからである。C では残念ながら大きな影響はない。

^{*16} 今回は while を用いたが、本来は for で実装すべき操作である。

```

int main(void) {
    for (int i = 0; i <= 180; i += 20) {
        const double angle = i * M_PI / 180.0;
        const double cosval = cos(angle);
        printf("cos(%lf) is %lf\n", angle, cosval);
        if (cosval < 0.0) {
            break;
        }
    }
    return 0;
}

```

2.2.7 continue 文

ある条件を満たした場合に `for` や `while` の先頭に戻り、次の繰り返しの進みたい場合には、`continue` を用いる (より正確には、`continue` から後を実行せずに、次の条件判定を行う)。例 2.2.5. では、割り切れない場合には `continue` を実行してすぐに `i <= number` の判定を行っている。割り切れる場合は `printf(...)` を実行してから `i <= number` の判定を行っている。ちなみに、このプログラムは 80 の全ての約数を出力するものである。

例 2.2.5.

```

#include <stdio.h>

int main(void) {
    const int number = 80;
    for (int i = 1; i <= number; ++i) {
        /* 'a % b' yields the residual of a/b */
        const int residual = number % i;
        if (residual != 0) {
            continue;
        }
        printf("%d is a measure(YAKUSUU) of %d\n", i, number);
    }
    return 0;
}

```

練習 2.2.1. `if` 文の練習として、 $ax + b = 0$ の解を求めるプログラムを書け。 a , b , x を表示させて終了させること。また、 a , b はあらかじめプログラムの中で決めてよい。

練習 2.2.2. `for` あるいは `while` の練習として、 $n!$ (階乗) を求めるプログラムを書け。 n と結果を表示させて終了させること。また、 n はあらかじめプログラムの中で決めてよい。

練習 2.2.3. 2 から 100 までの素数を表示するプログラムを書け。

2.3 配列

2.3.1 1次元配列

1次元配列を用いるときには、

```
int a[10];
```

のように宣言する。この場合、

```
a[0], a[1], ..., a[9]
```

の10個の要素を持つ配列が生成される。添字が0から始まっていること、定義した時点では初期化がされていない(何らかの値がすでに入っていることがある)ことに注意せよ。例2.3.1.では、`#define`によって `N_ELEMENT` を10と定義している^{*17}。for文によって、配列 `array` に数字が格納されている。あとはそれを順番に表示して終了する。なお、

```
int n = 10;
int a[n];
```

はコンパイルエラーとなる。この記法を使うときは、配列のサイズを動的(プログラム実行時)に決めることはできない^{*18}。入力値などに応じてサイズを動的に決めたい場合は、`malloc` と `free` (2.12節)を用いる。

なお、大きすぎる配列(概ね8MB以上が目安)を確保するとクラッシュすることがあるので注意せよ。

例 2.3.1.

```
#include <stdio.h>

#define N_ELEMENT 10

int main(void) {
    int array[N_ELEMENT];
    for (int i = 0; i < N_ELEMENT; ++i) {
        array[i] = i * i * i;
    }
    for (int i = 0; i < N_ELEMENT; ++i) {
        printf("array[%d] = %d\n", i, array[i]);
    }
    return 0;
}
```

2.3.2 2次元配列

2次元配列は、

```
int a[2][3];
```

のように宣言する。この場合、

```
a[0][0], a[0][1], a[0][2]
a[1][0], a[1][1], a[1][2]
```

^{*17} 詳しくは、参考書などの「プリプロセッサ」に関する説明を参照のこと。`N_ELEMENT` はプログラムの実行中に変化する数ではなく、コンパイル時に定まっている定数である。

^{*18} 1999年改訂のC標準規格C99では認められているが、2011年改訂のC11では必須機能でなくなってしまった。

の六つの要素を持つ 2 次元配列が生成される。例 2.3.2. は、 2×2 の行列の逆行列を求めるものである。

例 2.3.2.

```
#include <stdio.h>

int main(void) {
    const double matrix[2][2] = { { 10.0, 5.0 }, { 8.0, 14.0 } };
    /* matrix =
    | 10.0 5.0 |
    | 8.0 14.0 | */
    const double det = matrix[0][0] * matrix[1][1]
        - matrix[0][1] * matrix[1][0];
    if (det == 0.0) {
        printf("inverse matrix does not exist\n");
    } else {
        const double inverse[2][2]
            = { { matrix[1][1] / det, -matrix[0][1] / det },
                { -matrix[1][0] / det, matrix[0][0] / det } };
        printf("inverse[0][0] = %lf\n", inverse[0][0]);
        printf("inverse[0][1] = %lf\n", inverse[0][1]);
        printf("inverse[1][0] = %lf\n", inverse[1][0]);
        printf("inverse[1][1] = %lf\n", inverse[1][1]);
    }
    return 0;
}
```

初期化は括弧を用いることでまとめて行うことができる。

練習 2.3.1. int 型で大きさが 5 の 1 次元配列 a と b を準備し、配列 a にあらかじめ数字を代入しておく。その配列 a の要素をすべて配列 b に代入し、その後、配列 a をすべて 0 にするプログラムを書きなさい。a と b をすべて表示させて終了すること。

練習 2.3.2. double 型で大きさが 2×2 の 2 次元配列 a と b を準備し、その積を計算するプログラムを書きなさい。a と b とその積を表示させて終了すること。ただし、a と b はあらかじめプログラムの中で決めてよい。

2.4 文字列と標準入力

文字列の取り扱いは非常に複雑である。ここではごく基本的なことに限定して解説する。また、標準入力 (キーボードなどからの入力) を受け付け、それをプログラムで利用する方法についても説明する。

2.4.1 文字列

文字列とは、名前通り文字の配列である。文字列を扱うためには、`char s[10];` のように宣言する。ただし、次のような形の代入はできない。

```
char s[10];
s = "Hello";
```

文字列の宣言と同時に文字を代入するには次のように行う。

```
char s[10] = "Hello";
```

あるいは、

```
char s[] = "Hello";
```

後者は自動的に [] の中は 6 になる。(前者の 10 の部分は適当な数字でよいが、Hello 5 文字を記憶するためには 6 以上の数字を用いる必要がある。) では、なぜ [] の中は 6 になるのか? なぜ 5 でないのか? という疑問が生じるだろう。これには以下のような事情が関係している。

文字列を使う場合、どこでその文字列が終了するかを判断できなければならない。ところが、ここで使っている s という変数は単にその文字列の先頭を示しているだけで、どこで終わるかという情報を持っていない(2.5.3 節参照)。そのため、C 言語では文字列の最後に必ず “\0” という特殊な文字をつけるという決まりがある。そうすることで、文字列の最後を判断することが可能になるわけだ。このような理由で、5 ではなく 6 になるのである。

<pre>char s[] = "Hello";</pre> <table border="1" style="margin: 10px auto; text-align: center; border-collapse: collapse;"> <tr> <td style="padding: 2px 10px;">H</td> <td style="padding: 2px 10px;">e</td> <td style="padding: 2px 10px;">l</td> <td style="padding: 2px 10px;">l</td> <td style="padding: 2px 10px;">o</td> <td style="padding: 2px 10px;">\0</td> </tr> </table> s[0]s[1]s[2]s[3]s[4]s[5]	H	e	l	l	o	\0	<pre>char s[10] = "Hello";</pre> <table border="1" style="margin: 10px auto; text-align: center; border-collapse: collapse;"> <tr> <td style="padding: 2px 10px;">H</td> <td style="padding: 2px 10px;">e</td> <td style="padding: 2px 10px;">l</td> <td style="padding: 2px 10px;">l</td> <td style="padding: 2px 10px;">o</td> <td style="padding: 2px 10px;">\0</td> <td style="padding: 2px 10px;"></td> <td style="padding: 2px 10px;"></td> <td style="padding: 2px 10px;"></td> <td style="padding: 2px 10px;"></td> </tr> </table> s[0]s[1]s[2]s[3]s[4]s[5]s[6]s[7]s[8]s[9]	H	e	l	l	o	\0				
H	e	l	l	o	\0												
H	e	l	l	o	\0												

2.4.2 標準入力 (1) : gets 関数

例 2.4.1. を作成しコンパイルしてみよう (コンパイラによっては、「gets は危険だ」と警告が出るかもしれないが、実行ファイルはちゃんとできているはずである)。これを実行すると、Input: と表示され入力を促される。ここで、uni などと入力すると、uni と表示されてプログラムが終了する。この場合、gets という関数によって入力された文字列が str にコピーされる。注意すべきことは、str が str[20] と宣言されているので入力すべき文字列は 19 文字以内に限られるということである。しかし実際には、ユーザーは 20 字以上入力することができる。その場合、はみ出た文字は用意された領域の外に書き込まれる。もしそこが、別の変数用に使われていたら? これはとても危険なことである。gets を使う場合は注意が必要である^{*19}。

例 2.4.1.

```
#include <stdio.h>

int main(void) {
    char str[20];
    printf("Input: ");
    gets(str);
    printf("%s\n", str);
    return 0;
}
```

2.4.3 標準入力 (2) : scanf 関数

例 2.4.1. ではたとえ数字 (19 桁以下) を入力してもそれは文字列として扱われるため、数字として足し算などに用いるためには atoi などの関数を用いる必要がある。別の方法として、scanf を用いて入力した数字をそのまま数字として扱う方法を例 2.4.2. に示す (ソースコード中の &i や &x についている & については後述する)。

^{*19} どうしても gets が使用したいときにはより安全性が高い fgets を使用せよ。

例 2.4.2.

```
#include <stdio.h>
#include <string.h>

int main(void) {
    int i;
    printf("Input(int): ");
    scanf("%d", &i);
    printf("%d\n", i);
    double x;
    printf("Input(double): ");
    scanf("%lf", &x);
    printf("%lf\n", x);
    return 0;
}
```

練習 2.4.1. 練習 2.2.2. で、標準入力から n を決定し、その答えを表示するプログラムを書きなさい。

2.5 ポインタ

ポインタの習得は C 言語の習得の中で一つの大きな壁である。はじめは訳がわからないかもしれないが、C 言語の基本的な部分の一つなので、いろいろな例に触れて慣れてほしい。慣れが一番である。

2.5.1 とりあえず (1)

例 2.5.1. を作成し実行してみよう。上手くいけば、“q is 200 and *p is 200.” と表示されるはずである。この例で出てくる p がポインタである。int 型のポインタを宣言するには、

```
int *p;
```

のように*をつける。人によっては、

```
int* p;
```

と宣言したほうがイメージしやすいかもしれない。つまり、p は int* 型 (int のポインタ) ということである (しかし、2 個以上のポインタを宣言するためには、int *p, *q; としなければならない。int* p, q; とすると q は int 型になってしまう)。

例 2.5.1. の q に格納されている 200 はコンピュータのメモリーのどこかに電氣的に存在するはずである。その格納場所を示すものがアドレス (言うなればメモリー上の住所) であり、これを &q で表す。今回はこれを p へと代入したので、p は q が存在する場所を指し示すようになる。

変数からそのアドレスを抽出する&と対照的なのが*であり、これは逆にポインタからその指している先そのものを抽出する*²⁰。p はあくまで q を指し示しているだけ (q が存在するアドレスの情報しか持っていない) なので、実体 q を利用する際には*p とせねばならないのである。

例 2.5.1.

```
#include <stdio.h>

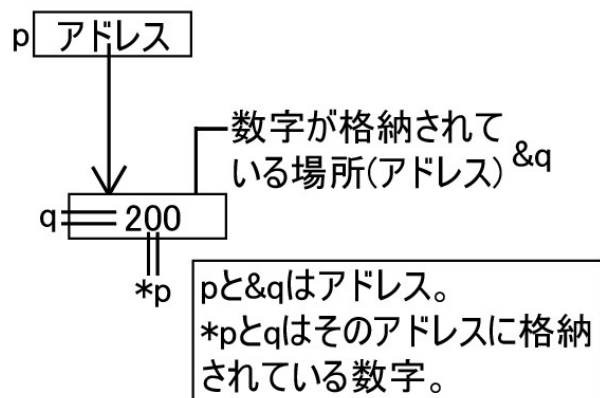
int main(void) {
    /* q を初期化 */
```

²⁰ *は型から 1 つアスタリスクを剥がし、逆に&は 1 つ増やす、と覚えるとよい。

```

int q = 200;
/* q のアドレスを p に代入する */
int* p = &q;
/* p から指している対象そのもの (q) を取り出す */
printf("q is %d and *p is %d.\n", q, *p);
return 0;
}

```



2.5.2 とりあえず (2)

では、もう一つ例を示そう。例 2.5.2. では、`p = &q;` によりポインタ代入を行ったので `p` というポインタで `q` にアクセスできる。ここでは、`q` に値を代入するかわりに、`*p` を用いて値を代入している。`p` はポインタで、`*p` は `p` が指している対象である `q` そのものである。実行結果は、“`q is 300 and *p is 300.`”となる。

例 2.5.2.

```

#include <stdio.h>

int main(void) {
    int q;
    int* p = &q;
    /* 値を取得するだけでなく書き換えることもできる */
    *p = 300;
    printf("q is %d and *p is %d.\n", q, *p);
    return 0;
}

```

記号`*`、`&`などの意味と役割をもう一度復習しておこう。

```

int *p;
int q;

```

の時、

`p` ポインタ (つまりどこかのアドレス)
`*p` 実体
`q` 実体
`&q` `q` のアドレス

である。

注意すべきことは、例 2.5.1. や例 2.5.2. で `p = &q;` がない場合は `p` がどこを指しているか未定義であるということだ。したがって、事前の初期化なしに `*p = 300;` のように書き換えを試みたり、`printf` 内で `*p` を表示させるのは危険である。何が起るかわからない。

この節では、静的に宣言された変数を指すためにポインタを用いてきたが、より高度な使い方については 2.12 節で紹介する。

2.5.3 ポインタと配列

ポインタと配列には密接な関係がある。

```
int array[10];
```

と宣言した場合、`array[0]` や `array[5]` など配列の要素にアクセスすることができた。実は、`array` だけでも意味を持つ。`array` は配列型 (`int[10]` 型) の変数で、プログラムコード中に登場した時は、大抵の状況では、自動的に“配列の先頭要素のアドレス”に暗黙的に変換されて解釈される。つまり、`array` は `array[0]` へのポインタとして解釈され、`*array` は `array[0]` を意味する。さらに、ポインタに整数を足すとその数だけ先の要素を指すようになる。例えば `array + 2` は `array[2]` へのポインタであり、`*(array + 2)` は `array[2]` と等価である。

では、例 2.5.3. を見てみよう。`strlen` という関数は `string.h` 内で宣言されている文字列の長さを返す関数である。最初の `for` 文は文字型の配列 `str` の中身を順番に書き出している。次の `for` 文がポイントである。ここではまず文字型のポインタ `p` に文字型の配列 `str` の先頭ポインタ `str` を代入している。そして、文字型の実体である `*p` が `\0` でない場合はその `*p` を表示し、ポインタ `p` の指す部分を一つ進めている。これを `*p` が `\0` になるまで繰り返すことで、最初の `for` 文と同じ結果を表示できるというわけである。

例 2.5.3.

```
#include <stdio.h>
#include <string.h>

int main(void) {
    const char str[] = "ABCDE";
    const int num = strlen(str);
    for (int i = 0; i < num; ++i) {
        printf("%c", str[i]);
    }
    printf("\n");
    for (const char* p = str; *p != '\0'; ++p) {
        printf("%c", *p);
    }
    printf("\n");
    return 0;
}
```

2.5.4 const とポインタについて

ポインタを介して変数を操作出来るのは便利な反面、意図せず書き換えてしまった結果バグを生じたり、そもそも `const` のついた変数をどうやってポインタで参照するのか (うっかり書き換えてしまったりしないのか) といった問題が生じる。これを解決するのが `const` つきのポインタである。

型	ポインタ自体を変更できるか	そのポインタから実体を変更できるか
type*	○	○
const type*	○	×
type* const	×	○
const type* const	×	×

少し難しい概念ではあるが、バグを防ぐ上で非常に強力であるので困ったときにはぜひ使ってみてほしい。

練習 2.5.1. 次のプログラムの `printf` の部分を配列ではなく、ポインタを使ったものを書き換えなさい

(array を使っても、あるいは、`int *p;` を加えてもよい)。

```
#include <stdio.h>

int main(void)
{
    int array[10];
    for (int i = 0; i < 10; ++i) {
        array[i] = i * i;
    }
    for (int i = 0; i < 10; ++i) {
        printf("array[%d] = %d\n", i, array[i]);
    }
    return 0;
}
```

2.6 複素数

量子力学などの数値計算では複素数を用いることも多い。C においても複素数変数を扱うための型が用意されている^{*21}。

```
float complex    単精度複素数型
double complex   倍精度複素数型
```

複素数型を使うためには、プログラムの冒頭で `complex.h` ヘッダファイルを読み込む必要がある。`csin`, `cexp`, `clog` などの複素初等関数も用意されている^{*22}。以下の例では、 $\exp(i\pi)$ の計算を行っている。

例 2.6.1.

```
#include <complex.h>
#include <math.h>
#include <stdio.h>

int main(void) {
    const double complex x = 0 + 1 * I; /* 虚数単位 */
    const double complex y = cexp(x * M_PI);
    printf("i = (%lf,%lf)\n", creal(x), cimag(x));
    printf("e^{i*pi} = (%lf,%lf)\n", creal(y), cimag(y));
}
```

^{*21} 複素数の機能は C11 で必須でなくなってしまった。

^{*22} 単精度複素数の場合は、`csinf`, `cexpf`, `clogf` のように関数名の後に `f` を付ける。

```
    return 0;
}
```

虚数単位は大文字の `I` と書く。あるいは、大文字の `CMPLX` 関数^{*23}を使って、`x = CMPLX(0,1)` のように表すこともできる。複素数の実部、虚部を取り出すにはそれぞれ、`creal`, `cimag` を使う。絶対値、偏角、複素共役はそれぞれ、`cabs`, `carg`, `conj` である^{*24}。

■`tgmath.h` について

C 言語では関数名に重複があってはならないので、数学関数が型ごとに違う名前を持っている。これをいちいち切り替えるのは面倒であるが、新しい規格においては勝手に引数の型を読み取って適切な関数を呼び出してくれる便利な `tgmath.h` というものが追加されている。

2.7 関数

2.1 節で C 言語の基本構造を説明したが、`main()`... だけでプログラムを書くことはほとんどない。大きなプログラムになればなるほど関数化を行い、役割分担をはっきりさせるのがよい。

2.7.1 あまり良くない例

例 2.7.1. は円の面積の計算するために直接その公式を書いている。しかし、何度も何度も円の面積を計算したいとき、半径を入力すれば面積が返ってくるような関数があれば非常に便利である。

例 2.7.1.

```
#include <math.h>
#include <stdio.h>

int main(void) {
    const double radius = 2.0;
    const double area = radius * radius * M_PI;
    printf("Radius: %lf, Area: %lf\n", radius, area);
    return 0;
}
```

2.7.2 関数化

では、例 2.7.1. を例 2.7.2. のように変更してみよう。このような小さなプログラムでは関数化の効力は乏しいが、大きくなればなるほど、また複雑になればなるほど、その効力は絶大になる。例 2.7.2. では、`circle_area` という関数が定義されている。この関数では、`double` 型の値を引数として受け取り、面積を計算してその値を戻り値にしている。

例 2.7.2.

```
#include <math.h>
#include <stdio.h>

double circle_area(double r) {
    return r * r * M_PI;
}
```

^{*23} 単精度複素数の場合、`CMPLXF`。

^{*24} 単精度複素数の場合、それぞれ `crealf`, `cimagf`, `cabsf`, `cargf`, `conjf` である。

```
int main(void) {
    const double radius = 2.0;
    const double area = circle_area(radius);
    printf("Radius: %lf, Area: %lf\n", radius, area);
    return 0;
}
```

通常、関数は使う前に“宣言”する必要がある。(例 2.7.2. は特殊な場合であり、関数の定義が宣言を兼ねている) きちんと宣言を行うときには、例 2.7.3. のように関数の引数名と処理部分がない不完全な定義のようなもの(これが宣言の書式である)を書く必要がある。

関数を使用する際には使用箇所から宣言が見えるようにしておく必要があるが、逆に宣言さえ見えていれば定義はどこにあっても構わない^{*25}。例 2.7.3. は、“宣言は見えるが、定義は見えない” 場合の典型例になっている。

例 2.7.3.

```
#include <math.h>
#include <stdio.h>

double circle_area(double);

int main(void) {
    const double radius = 2.0;
    const double area = circle_area(radius);
    printf("Radius: %lf, Area: %lf\n", radius, area);
    return 0;
}

double circle_area(double r) {
    return r * r * M_PI;
}
```

2.7.3 ポインタを引数にする関数

例 2.7.2. では関数の戻り値が面積の一つだけであった。しかし、例えば割り算で商と余りを返したい場合、例 2.7.2. のような関数ではうまく実現できない。なぜなら、戻り値は 1 個しか指定できないからである。これを解決するには、ポインタを引数とする関数を作り、あらかじめ準備しておいた答えを入れる変数を関数の引数にポインタとして渡し、関数内でそこに答えを入れてもらって関数の外で受け取るという方法が有効である。重要な点は、ポインタでなければ関数内で代入した値を関数の外で使うことはできないということである。

例 2.7.4. では、まず main で答えを入れてもらうための shou と amari という箱 (変数) を準備している。次に、そのポインタ (アドレス) を関数 division に渡している。そして、division 内で答えを代入してもらい、関数の外で printf を用いて答えを表示している。division という関数の先頭にある void という型は (戻り値の形では) 何も返さないことを示すためのものである。

例 2.7.4.

```
#include <stdio.h>

void division(int dividend, int divisor, int* const quotient,
              int* const residual) {
    *quotient = dividend / divisor;
    *residual = dividend % divisor;
}
```

^{*25} より厳密に言うならば、gcc に渡す .c ファイルのどれか、あるいはリンクするライブラリに定義が含まれていれば問題ない。

```

    }

    int main(void) {
        const int josuu = 3;
        const int hi_josuu = 13;
        int shou, amari;
        division(hi_josuu, josuu, &shou, &amari);
        printf("%d / %d = %d ... %d\n", hi_josuu, josuu, shou, amari);
        return 0;
    }

```

ポインタを使う理由がはっきりと分からない場合は、例 2.7.5. を作って実行してみてほしい。間違った答えが表示されるだろう。なぜなら、この場合 `division` という関数では、`quotient` と `residual` という二つの一時変数が生成され、それらに計算結果が代入された直後に破棄されるからである。一時変数は関数の処理が終了すると同時に破棄され、値は `main` の中の `shou` と `amari` には引き継がれない^{*26}。このため、関数に外部から値を渡すだけであれば、例 2.7.5. のような引数の宣言方法でよい（「値渡し」と呼ぶ）が、内での計算結果を関数の外で使いたい場合は、ポインタを使って変数のアドレスを渡す（「ポインタ渡し」と呼ぶ）必要がある。

例 2.7.5.

```

#include <stdio.h>

void division(int dividend, int divisor, int quotient, int residual) {
    quotient = dividend / divisor;
    residual = dividend % divisor;
}

int main(void) {
    const int josuu = 3;
    const int hi_josuu = 13;
    int shou, amari;
    division(hi_josuu, josuu, shou, amari);
    printf("%d / %d = %d ... %d\n", hi_josuu, josuu, shou, amari);
    return 0;
}

```

2.7.4 関数ポインタ

ポインタはメモリ上にある変数を指し示すものであったが、実は関数そのものもメモリ上に配置されているのでポインタが使える。例えば `int` を受け取って `double` を返す関数用のポインタ `p` を宣言したいときには `double (*p)(int);` と書けばよい^{*27}。

関数ポインタを使うと“関数を引数として受け取る関数”^{*28}が実現でき、うまく使うと非常に強力である。

2.7.5 Fortran の関数や手続きの利用

LAPACK^{*29}などの多くのライブラリが Fortran で書かれている。これらと同等の機能を持つ関数を C や C++ で作ることもできるが、既に存在するのであればそちらを使った方が便利である。ここでは、C 言語の中から Fortran の関数や手続きを呼ぶ方法を簡単に紹介する。

Fortran の関数や手続きの名前を C 言語から呼ぶためには、その名前をすべて小文字にして、最後に `_`（下

^{*26} ブロック中での変数定義の挙動を思い出そう。

^{*27} 変数宣言らしからぬ特殊な記法であるが、これであっている。

^{*28} いわゆる汎関数のようなものを作ることができる。

^{*29} 行列の対角化、連立 1 次方程式の求解など線形計算を行うライブラリ。ほぼ全ての計算機で利用可能である。

線) を付けなければならない。また、関数などの引数の型も適切に読み変える必要がある。例えば、

例 2.7.6.

```
Real*8 CALC(I, X, Y)
Integer*4 I
Real*4 X
Real*8 Y
```

という Fortran の関数が存在するとする。このとき C では以下のような宣言を書く必要がある。

例 2.7.7.

```
extern double calc_(int*, float*, double*);

int main(void) {
    int i = 10;
    float x = 20.0F;
    double y = 30.0;
    double ret = calc_(&i, &x, &y);
    return 0;
}
```

全ての引数をポインタ渡しとしなければならないことに特に注意せよ。コンパイルは C (gcc) と Fortran (gfortran) で別々に行い、最後にリンクすればよい^{*30}。

2.8 構造体

データ構造を取り扱うためには構造体を用いる。

例 2.8.1. を見てほしい。個人のデータを取り扱うために `personal_data` という一つの箱を準備している。これが構造体である。もし構造体を使わなければ、同じ内容のプログラムを実現するために複数の配列を準備する必要が出てくる。これは見た目にも格好悪いし、拡張性も乏しく複雑になる。例 2.8.1. では、`struct personal_data pdata;` で `pdata` を構造体とし宣言している。少し分かりにくいかもしれないが、`int n;` と比べてみると、`int` と “`struct personal_data`” が同じ立場であって、`int` と “`struct`” が同じ立場というわけではないことに注意してほしい。`int` という型はあらかじめ C 言語で決められた型なのに対し、構造体は自分が好きなように使いやすいように決めた型だと考えても構わない。つまり、“`struct personal_data`” 型というものを自分で作ったと考えるのである。そうすれば、`struct personal_data pdata;` という使い方も理解できるはずである。

構造体内の各データには、ドット演算子 (.) を使ってアクセスする。また、年齢を文字列 `buffer[16]` として受け取っているため、`atoi` 関数を用いて数字に変換している。`atoi` は文字列を整数に変換する関数である。詳しくはコマンドラインで `man atoi` としてみよ。

例 2.8.1.

```
#include <stdio.h>
#include <stdlib.h>

struct personal_data {
    char family_name[16];
    char given_name[16];
    int age;
};
```

^{*30} LAPACK はすでにコンパイル済みなので `gfortran` は必要ない。リンクの方法については 2.14.2. を見よ。

```

int main(void) {
    struct personal_data pdata;
    char buffer[16];
    printf("Input family name: ");
    gets(pdata.family_name);
    printf("Input given name: ");
    gets(pdata.given_name);
    printf("Input age: ");
    gets(buffer);
    pdata.age = atoi(buffer);
    printf("Family Name = %s\n", pdata.family_name);
    printf("Given Name = %s\n", pdata.given_name);
    printf("Age          = %d\n", pdata.age);
    return 0;
}

```

構造体を指すポインタの場合は、次の例 2.8.2. のように、アロー演算子 (->) でその構造体の要素にアクセスすることができる。例題中の `qdata->p[0]` は `(*qdata).p[0]` と等価である。ちなみに、この例の構造体は粒子の質量と運動量を格納している。

例 2.8.2.

```

#include <stdio.h>

struct particle_data {
    double mass;
    double p[3]; /* Momentum */
};

int main(void) {
    const struct particle_data pdata = { 0.14, { 1.2, 1.3, 1.4 } };
    const struct particle_data* const qdata = &pdata;
    printf("Mass = %lf\n", qdata->mass);
    printf("Momentum = (%lf, %lf, %lf)\n", qdata->p[0], qdata->p[1],
        qdata->p[2]);
    return 0;
}

```

練習 2.8.1. “月”, “日”, “1 月 1 日からの日数 (うるう年ではない)” を構成要素とする構造体を作って、“月” と “日” を標準入力から入力すれば、自動的に “1 月 1 日からの日数” の部分を埋め、最後にそれを表示して終了するプログラムを書きなさい。

2.9 ファイルの取り扱い

ファイルから数字を読みこんだり、ファイルに結果を書き込んだりするプログラムを紹介する。ここで挙げた例だけでも基本的なことはできるはずである。より詳しく知りたい場合は参考書などを参照してほしい。

2.9.1 ファイルから数字の読み込み

例 2.9.1. がファイルから数字を読み込むときの雛型となる。まず、`FILE *fp;` でファイルを取り扱うための構造体を宣言する。`char *filename` の部分でファイル名を宣言する。そして、`fopen` でファイルを開く。 (“r” は read (読み取り専用) でファイルを開くことを示す。) `if (fp==NULL)` の部分はエラーが起きたとき

に処理され、プログラムを強制終了させる^{*31}。fscanf で順番にファイルに書かれている数字を double 型で読み取っている。fscanf の使い方は、

```
fscanf(ファイル構造体のポインタ, "フォーマット", &変数 1, &変数 2, .....);
```

で、ファイル構造体のポインタ部分以外は scanf と同じである。最後に fclose(fp); でファイルを閉じている。

例 2.9.1.

```
#include <stdio.h>
#include <stdlib.h>

double product(double x1, double y1, double x2, double y2) {
    return x1 * x2 + y1 * y2;
}

int main(void) {
    const char filename[] = "vectors.txt";
    FILE* fp = fopen(filename, "r");
    if (fp == NULL) {
        printf("Can't open file %s\n", filename);
        exit(1);
    }
    double x1, y1, x2, y2;
    /* read first line */
    fscanf(fp, "%lf %lf\n", &x1, &y1);
    /* read second line */
    fscanf(fp, "%lf %lf\n", &x2, &y2);
    fclose(fp);
    const double p = product(x1, y1, x2, y2);
    printf("Product of (%lf,%lf) and (%lf,%lf) is %lf\n", x1, y1, x2, y2, p);
    return 0;
}
```

例 2.9.1. を実行するためには、vectors.txt というファイル名で以下のような内容の入力ファイルも作っておく必要がある。

```
1.0    2.0
-1.5   3.5
```

例 2.9.1. のデータを読み込む部分を、例 2.9.2. のように while を使って書き直してみよう。こちらの方がファイルに存在するデータを最後まで読み取るという点で、より汎用性が高い。fscanf はデータの読み込みに失敗すると EOF を返すので、EOF が返ってくるまで読み込みを繰り返している^{*32}。

例 2.9.2.

```
#include <stdio.h>
#include <stdlib.h>

#define N_DATA 100

int main(void) {
    const char filename[] = "vectors.txt";
```

^{*31} NULL は特別なポインタで、“なにか異常が起きた結果、ポインタが有効な場所を指さずに宙ぶらりんになっている”ことを表現するためにある。今回の例ではファイルが何らかの原因で読み取れなかったときに NULL が返ってくる。

^{*32} while の条件式チェックと読み込みを同時に行っている。


```

FILE* fp = fopen(filename, "r");
if (fp == NULL) {
    printf("Can't open file %s\n", filename);
    exit(1);
}
double x[N_DATA][2];
int index = 0;
/* read data */
while (fscanf(fp, "%lf %lf\n", &x[index][0], &x[index][1]) != EOF) {
    printf("Data %d : (%lf, %lf)\n", index, x[index][0], x[index][1]);
    ++index;
}
fclose(fp);
return 0;
}

```

2.9.2 ファイルへの数字の書き出し

例 2.9.3. がファイルへ数字を書き出すときの雛型である。ファイルを開くところまでは例 2.9.1. とはほぼ同じで、違う点は `fopen` の第二引数が `"r"` ではなく `"w"` (書きこみ) である点である。書き出すときには `fprintf` という関数を使っている。`fprintf` の使い方は、

```
fprintf(ファイル構造体のポインタ, "フォーマット", 変数 1, 変数 2, .....);
```

で、ファイル構造体のポインタ部分以外は `printf` と同じである。最後に、`fclose(fp);` でファイルを閉じる。

例 2.9.3.

```

#include <math.h>
#include <stdio.h>
#include <stdlib.h>

double circle_area(double r) {
    return r * r * M_PI;
}

int main(void) {
    const char filename[] = "circle_area.txt";
    FILE* fp = fopen(filename, "w");
    if (fp == NULL) {
        printf("Can't open file %s\n", filename);
        exit(1);
    }
    for (int i = 1; i <= 10; ++i) {
        const double radius = i;
        const double area = circle_area(radius);
        fprintf(fp, "%lf -> %lf\n", radius, area);
    }
    fclose(fp);
    return 0;
}

```

2.10 その他の制御文

`for`, `while` 以外の制御文の書式を紹介する。

2.10.1 switch-case 文

1 の場合には A を、2 の場合には B を、3 の場合には C を、という風に条件分岐をしたい場合に、**switch-case** 文を用いる。

例 2.10.1.

```
#include <stdio.h>
#include <string.h>

int main(void) {
    int i;
    printf("Input(int): ");
    scanf("%d", &i);
    printf("%d / 3 no amari ha ", i);
    switch (i % 3) {
        case 1:
            printf("1 desu.\n");
            break;
        case 2:
            printf("2 desu.\n");
            break;
        default:
            printf("0 desu.\n");
    }
    return 0;
}
```

今回は `i % 3` をチェックの対象とし、上から 1 のとき、2 のとき、それ以外の順で処理を記述している。もし例 2.10.1. の **switch-case** 文中に `break;` がないと、`case 1:` の場合は `case 2:` の部分も `default:` の部分も実行してしまうことに注意^{*33}。(`case 1:` 以下の部分から下の部分をすべて実行する。) したがって、`case 2:` の直前で抜けるためには `break;` が必要となる。実際に `break;` を削除して試してみよう。この場合は常に `...0 desu.` が表示されるはずである。

switch-case 文を簡単にまとめると、次のようになる。

```
switch (X) {
    case A: ブロック 1
    case B: ブロック 2
    ...
    default: ブロック N
}
```

switch 文に処理が到達すると、`X` が上から順に `case` と照合されていき、一致したところでブロックに入る。**switch** から抜けるには `break;` を使用する。`case` はいくつあっても構わない。

2.10.2 do-while 文

while 文と同様に、ある条件が満たされている場合に繰り返しを行うために **do-while** 文を使う。**while** 文と違う点は、条件の判定する“タイミング”である。**while** 文の場合は、まず条件を判定し、満足していれば **while** 内を実行するが、**do-while** 文の場合は、まず **do-while** 内を実行してから条件を判定する。したがって、**while** 文の場合は **while** 内を 1 度も実行しない場合があるが、**do-while** 文の場合は必ず 1 度は

^{*33} -Wextra をつけていればきちんと警告が出るので安心してよい。

do-while 内を実行する。

例 2.10.2.

```
#include <stdio.h>

int main(void) {
    int i = 100;
    int sum = 0;
    do {
        sum += i;
        --i;
    } while (i != 0);
    printf("sum of integers from 100 to 1 is %d\n", sum);
    return 0;
}
```

例 2.10.2. のように、while() の後ろの ; を忘れないこと。do-while 文の基本的な動作は次のようになる。

```
do {
    ブロック
} while (A);
```

do に処理が到達すると、“ブロックを実行 → A をチェックし、偽なら抜ける”の操作が繰り返される。

2.11 コマンドライン引数の受け渡し

main 文には、int main(void) 以外に、int main(int, char**), あるいは int main(int, char*[]) という書き方がある。これらの書式を利用すれば、実行時に引数を与えてそれを利用することができる。つまり、今までは

```
$ ./a.out
```

として実行していたが、これを利用すれば、

```
$ ./a.out input.data output.data
```

のように、ファイル名や数値などをコマンドライン引数としてプログラムに渡すことができる。

2.11.1 ポインタ配列

新しい main 関数の説明の前に、引数の受け渡しに使われているポインタ配列に関して説明しておこう。ポインタ配列は、名前通り、ポインタを要素とする配列 (char*[]) である。配列はポインタを使ってアクセス可能なので、“ポインタのポインタ” (char**) と同様に使うことができる。

例 2.11.1.

```
#include <stdio.h>

int main(void) {
    char* name0 = "Alice";
    char* name1 = "Bob";
    char* name2 = "Claire";
    char* name3 = "David";
```

```

char* name[4];
name[0] = name0;
name[1] = name1;
name[2] = name2;
name[3] = name3; /* A */
/* B */
for (int i = 0; i < 4; ++i) {
    printf("Name%d : %s\n", i, name[i]);
}
/* C */
for (int i = 0; i < 4; ++i) {
    printf("Name%d : %c, %s\n", i, *(name + i), *(name + i));
}
return 0;
}

```

例 2.11.1. の結果は以下のようになる。

```

Name0 : Alice
Name1 : Bob
Name2 : Claire
Name3 : David
Name0 : A, Alice
Name1 : B, Bob
Name2 : C, Claire
Name3 : D, David

```

まず、`name[0]` の型は `char*` なので、`name0` の値などを代入することができる。A の時点で、`name` というポインタ配列の各要素にアドレスの代入したことになる。つまり、

```

name[0] = "Alice"という文字列の先頭アドレス
name[1] = "Bob"という文字列の先頭アドレス
name[2] = "Claire"という文字列の先頭アドレス
name[3] = "David"という文字列の先頭アドレス

```

となっている。したがって、B の `for` 文では、`printf` に文字列の先頭アドレスを渡すことで、名前を表示することができる。次に、C の `for` 文である。これは若干混乱を招くが、次が理解出来れば、何とかクリアできるだろう。

```

name = 文字へのポインタのポインタ = char** 型
*name = 文字へのポインタ = char* 型
**name = 文字 = char 型 (※ 文字列ではなく文字)

```

`printf` は `%s` で文字列の先頭アドレス、つまり、`char*` を受け取り、`%c` で文字型そのもの、つまり、`char` を受け取る。また、`name + i` という書き方は `&(name[i])` とほぼ等価である。

2.11.2 新しい main 関数

例 2.11.2. がコマンドライン引数を受け取ることのできる `main` 関数の例である。`int main(int argc, char* argv[])` は、`int main(int argc, char** argv)` と書いても同じである。好きなほうを使えばよい。また、伝統的に `argc` と `argv` という名前を使っているが、他の変数名でも構わない。

例 2.11.2.

```

#include <stdio.h>
#include <stdlib.h>

int main(int argc, char* argv[]) {
    if (argc != 3) {
        /* A */
        printf("Usage: sumXY InputFile OutputFile\n");
        exit(1);
    }
    const char* const InputFileName = argv[1];
    const char* const OutputFileName = argv[2];
    /* input */
    FILE* fp = fopen(InputFileName, "r");
    if (fp == NULL) {
        printf("Can't open file %s\n", InputFileName);
        exit(1);
    }
    int counter = 0;
    double x, y;
    double sumX = 0.0, sumY = 0.0;
    /* read data */
    while (fscanf(fp, "%lf %lf\n", &x, &y) != EOF) {
        sumX += x;
        sumY += y;
        ++counter;
    }
    fclose(fp);
    /* output */
    fp = fopen(OutputFileName, "w");
    if (fp == NULL) {
        printf("Can't open file %s\n", OutputFileName);
        exit(1);
    }
    /* write results */
    fprintf(fp, "Number of Data = %d\n", counter);
    fprintf(fp, "X Sum = %lf\n", sumX);
    fprintf(fp, "Y Sum = %lf\n", sumY);
    fclose(fp);
    return 0;
}

```

例 2.11.2. を 2.11.2.c という名前で保存し、次のようにコンパイルする。

```
$ gcc -o sumXY 2.11.2.c -Wall -Wextra
```

sumXY は例 2.11.2. の A の部分の printf のメッセージに合わせてある。次のようにして実行してみよう。

```
$ ./sumXY
Usage: sumXY InputFile OutputFile
```

このとき InputFile, OutputFile の二つのコマンドライン引数が必要であるというエラーが表示される。例 2.11.2. の A の部分を変更することで、エラーメッセージは変更することができる。次のように実行しても同じメッセージが出力されるはずである。

```
$ ./sumXY input.dat
```

```
$ ./sumXY input.dat output.dat 10
```

一番目の例は引数が足りず、二番目の例は引数が多すぎるからである。例 2.11.2. では `argc` が 3 以外ならエラーメッセージを表示するようにしてあるが、

```
$ ./sumXY input.dat output.dat 10
```

は 3 個の引数だからエラーが出るのはおかしい、と考えるかもしれない。しかし、これは正しい動作である。なぜなら、引数としてプログラム名 (`./sumXY`) も 1 個と数えられるからである^{*34}。つまり、二番目の例では、`argc` の値は 4 であり、`argv` の中身は

```
argv[0] = "./sumXY"の先頭アドレス
argv[1] = "input.dat"の先頭アドレス
argv[2] = "output.dat"の先頭アドレス
argv[3] = "10"の先頭アドレス
```

となっている。

`argc` が想定していた値と異なる場合、配列 `argv` の有効範囲からはみ出した部分を参照して致命的なバグを発生させることがあるので、`argc` のチェック機構は面倒でもつけるようにしたほうがよい。

最後に、1 行に 2 個の数字で 10 行程度書いた `input.dat` を作成し、次のように実行してみよう。`output.dat` に結果が書き込まれているはずである。

```
$ ./sumXY input.dat output.dat
```

2.12 動的な配列の確保: `malloc` と `free`

これまで、ポインタはあらかじめ確保した領域に対してのみ使ってきた。しかし、実行時に、入力値あるいは計算結果を反映して、50 個のデータを取り扱いたいときもあれば、100 個のデータを取り扱いたいときもある。もちろん、配列を利用してあらかじめ十分な大きさの領域 (いまの場合は 100 個以上の数) を確保しておけばよいかもしれないが、平均的に 10 個ぐらいしか利用しないのに、たまに 100 個の場合があるからといって常に 100 個分の領域を確保しておくことは、無駄のように思える。これを回避するために、動的に、つまり、実行時に領域を確保する手段がある。これが、`malloc` (**m**emory **a**llocation の略) である。

2.12.1 `malloc` の使い方

例 2.12.1. に `malloc` の使い方の雛型を示す^{*35}。`malloc` は領域を確保できた場合はその領域の先頭アドレスを返すが、領域が確保できなかった場合は `NULL` を返す。`NULL` の場合は、何らかのエラー処理を行う必要がある。例 2.12.1. では、プログラムを強制的に終了している。`sizeof` 関数は型の大きさ (バイト数) を調べる関数である。この場合は `int` の大きさ (通常 4) を調べている。`double` を大きさを調べたい

^{*34} `argv[0]` を用いると、例 2.11.2. の A のエラーメッセージは `printf("Usage: %s InputFile OutputFile\n", argv[0]);` と書くことができる。このようにしておくと、プログラム名をあらかじめソースコードに明示的に書き込む (「ハードコーディング」と呼ぶ) 必要がなくなる。

^{*35} 簡単のため、コマンドライン引数のチェックは行っていない。実際のプログラムでは、配列を確保する前に確保しようとしているサイズを確認すべきである。

場合は `sizeof(double)` のように使う。例 2.12.1. では、`int` の領域を `n_element` 個分確保したいので、`sizeof(int) * n_element` を `malloc` に与えている^{*36}。

例 2.12.1.

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char* argv[]) {
    const int n_element = atoi(argv[1]);
    int* array = (int*)malloc(sizeof(int) * n_element);
    if (array == NULL) {
        printf("Can't allocate memory.\n");
        exit(1);
    }
    for (int i = 0; i < n_element; ++i) {
        array[i] = i * i;
    }
    for (int i = 0; i < n_element; ++i) {
        printf("array[%d] = %d\n", i, array[i]);
    }
    return 0;
}
```

なお、コンパイル時に、`malloc` の部分で“型の不整合”という警告が出ることを防ぐために、`malloc` の返り値を明示的に `int*` 型に変換している。これをキャスト (型変換) という。

2.12.2 free の使い方

例 2.12.1. には (引数の数のチェック以外にも) 適切でない部分がある。`malloc` で確保した領域を解放していない点である。確保した領域を解放するためには、`free` という関数を使う^{*37}。具体的には、例 2.12.1. の `return 0;` の前に

```
free(array);
```

と 1 行書けばよい^{*38}。例 2.12.1. を若干変更し、`free` が重要となる例を考えよう。なお、今回は `assert.h` を用いて簡易的な `argc` のチェック機構をつけてある (`argc == 2` が満たされないとエラーメッセージを表示したのち異常終了する)。`assert` 機能はデバッグの際極めて重宝するので、覚えておくとよい^{*39}。

例 2.12.2.

```
#include <assert.h>
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char* argv[]) {
    assert(argc == 2);
    const int n_element = atoi(argv[1]);
    for (int j = 0; j < 3; ++j) {
        int* array = (int*)malloc(sizeof(int) * n_element);
```

^{*36} `malloc` はあらゆる型に対して使えるように要素数ではなくバイト数単位で必要メモリ量を指定する設計になっている。引数に 0 を与えてはならない。

^{*37} 例 2.12.1. のようにプログラムがすぐに終了してしまう場合には、`free` は必要ないと主張する人がいるかもしれない。その主張も確かに間違っている訳ではないが、本当に必要なときに忘れないために普段から書く癖をつけるべきである。

^{*38} `free` を同じ対象に対して複数回呼んではならない。

^{*39} もちろんチェックにはそれなりの計算コストがかかるが、`-DNDEBUG` をつけてコンパイルするとまとめて除去できるので安心してたくさん使ってほしい。

```

    if (array == NULL) {
        printf("Can't allocate memory.\n");
        exit(1);
    }
    for (int i = 0; i < n_element; ++i) {
        switch (j) {
            case 0:
                array[i] = i;
                break;
            case 1:
                array[i] = i * i;
                break;
            default:
                array[i] = i * i * i;
        }
    }
    for (int i = 0; i < n_element; ++i) {
        printf("array[%d] = %d\n", i, array[i]);
    }
    free(array);
}
return 0;
}

```

もし仮に `free` がない場合、`j = 1` のとき、`malloc` に成功すると `array` は新しく確保された領域の先頭アドレスを持つ。この時、`j = 0` で確保した領域はどうなったのだろうか？ その領域は、このプログラム自身が確保した領域として、このプログラムが終了するまで解放されない。しかも、`array` はすでに `j = 0` のときに確保した領域を忘れていたので、このプログラムからもその領域に適切にアクセスすることはできない。つまり、`j = 0` のときの領域はこのプログラムが持っているが、使うことができない領域として存在し続けることになる^{*40}。これは非常に無駄なことである。これを回避するためにも、使わなくなった領域は `free` することが重要である。もちろん、最近の計算機はメモリ（および、スワップ領域）をたくさん積んでいるので例 2.12.2. 程度のプログラムなら `free` がなくても平気で最後まで動くはずであるが、習慣として、`malloc` した領域を使い終わったら必ず `free` するのがよい。

2.12.3 動的な 2 次元配列

C 言語において、2 次元配列は 1 次元配列の先頭を指すポインタの配列として表すことができる。すなわち、各行をそれぞれ 1 次元配列として考え、各行の先頭を指すポインタを別の 1 次元配列に格納しておけばよい。この場合、後者の配列は要素がポインタ型の配列であるので、`double` 型の 2 次元配列の場合には

```
double** matrix;
```

と宣言する必要がある。その後、各行に対応する 1 次元配列を `malloc` する。

例 2.12.3.

```

#include <stdio.h>
#include <stdlib.h>

int main(void) {
    const int m = 3;
    const int n = 4;
    double** matrix = (double**)malloc(sizeof(double*) * m);
    if (matrix == NULL) {

```

^{*40} これを「メモリリーク」と呼ぶ


```

        printf("Can't allocate memory.\n");
        exit(1);
    }
    for (int i = 0; i < m; ++i) {
        matrix[i] = (double*)malloc(sizeof(double) * n);
        if (matrix[i] == NULL) {
            printf("Can't allocate memory.\n");
            exit(1);
        }
    }
    for (int i = 0; i < m; ++i) {
        for (int j = 0; j < n; ++j) {
            matrix[i][j] = (i + 1) * (j + 1);
        }
    }
    for (int i = 0; i < m; ++i) {
        for (int j = 0; j < n; ++j) {
            printf("matrix[%d][%d] = %lf\n", i, j, matrix[i][j]);
        }
    }
    for (int i = 0; i < m; ++i) {
        free(matrix[i]);
    }
    free(matrix);
    return 0;
}

```

この例では、 3×4 の 2 次元配列 (行列) を宣言している。まず、`matrix` に長さ 3 の 1 次元配列 (要素は `double*` 型) を確保する。さらに、各行を格納する長さ 4 の 1 次元配列 (要素は `double` 型) を 3 個確保し、その先頭アドレスを `matrix` に格納する。配列の (i, j) 成分へのアクセスは、静的な 2 次元配列の場合と同じく `matrix[i][j]` と書けばよい。`matrix[i][j]` は `*(*(matrix + i) + j)` と等価であることに注意せよ。`*(matrix + i)` により i 行目の先頭アドレスが得られ、それに j を足したアドレスの中身を参照することで、 (i, j) 成分が得られる。確保したメモリの解放の際は、まず各行に対応する 1 次元配列を解放した後、最後にそれらの先頭先頭アドレスを格納していた `double` ポインタ型の 1 次元配列を解放する必要がある^{*41}。

上記の方法で動的な 2 次元配列を取り扱えるようになるが、実際に行列として LAPACK などの数値計算ライブラリと組み合わせて使うには、二つの問題がある。一つ目は行列の要素の連続性の問題、二つ目は要素の並ぶ順番である。

LAPACK などの数値計算ライブラリでは、行列の要素は全てメモリ上で連続であると仮定されている。しかし、上記の方法では、各行のデータは連続であるが、行と行の間が連続であるとは限らない。C 言語では、連続した `malloc` の呼び出しで連続したメモリ領域が割り当てられることは保証されていないためである。メモリ上で `matrix[0][0]` の次には `matrix[0][1]`、`matrix[0][2]`、`matrix[0][3]` と並ぶが、その次が `matrix[1][0]` とは限らないのである。

この問題を解決するには、それぞれの行に対して `malloc` を行うのではなく、一度に $3 \times 4 = 12$ の長さの 1 次元配列をまとめて確保した上で、4 要素毎の要素 (各行の先頭に対応) のポインタをポインタ型配列に格納していけばよい。具体的には、コードを以下のように修正する。

例 2.12.4.

```

#include <stdio.h>
#include <stdlib.h>

int main(void) {

```

^{*41} 解放の順番を間違えるとエラーとなる。その理由を考えてみよ。

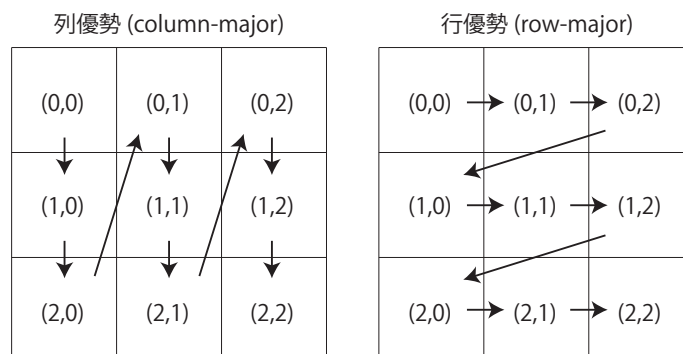
```

const int m = 3;
const int n = 4;
double** matrix = (double**)malloc(sizeof(double*) * m);
if (matrix == NULL) {
    printf("Can't allocate memory.\n");
    exit(1);
}
matrix[0] = (double*)malloc(sizeof(double) * m * n);
if (matrix[0] == NULL) {
    printf("Can't allocate memory.\n");
    exit(1);
}
for (int i = 1; i < m; ++i) {
    matrix[i] = matrix[i - 1] + n;
}
for (int i = 0; i < m; ++i) {
    for (int j = 0; j < n; ++j) {
        matrix[i][j] = (i + 1) * (j + 1);
    }
}
for (int i = 0; i < m; ++i) {
    for (int j = 0; j < n; ++j) {
        printf("matrix[%d][%d] = %lf\n", i, j, matrix[i][j]);
    }
}
free(matrix[0]);
free(matrix);
return 0;
}

```

修正前のコードと異なり、`malloc` と `free` は、それぞれ 2 回ずつしか呼び出されていないことに注意せよ。

次に、二つ目の問題、要素の並ぶ順番について考える。LAPACK などの数値計算ライブラリは、Fortran 言語を用いて書かれていることが多い。Fortran 言語では、2 次元配列 (行列) の要素は各列の要素がメモリ上で連続して並ぶ。例えば、 3×3 行列では、要素はメモリ上で、 $(0,0) \rightarrow (1,0) \rightarrow (2,0) \rightarrow (0,1) \rightarrow (1,1) \rightarrow (2,1) \rightarrow (0,2) \rightarrow (1,2) \rightarrow (2,2)$ の順に配置される。これを「列優勢 (column-major)」と呼ぶ。一方、C 言語では、メモリ上で `A[0][0]` の次に配置されるのは `A[0][1]` である。こちらは「行優勢 (row-major)」と呼ぶ。



C 言語上で行列の (i, j) 成分を `A[i][j]` に代入したもの (row-major) を LAPACK ライブラリに渡すと、LAPACK 側では要素が column-major で並んでいると解釈して計算を行う。すなわち、入力行列が転置されてしまう。また、計算結果も C 言語から見ると転置された状態で返されることになってしまう。

これを防ぐには、行列の (i, j) 成分を `A[i][j]` ではなく `A[j][i]` に格納するように決めてプログラムを書けばよいのだが、「常に順番を逆に書く」というのはプログラム作成時に混乱してしまう恐れがある。一つの

解決方法は、C プリプロセッサのマクロ機能を使い、ソースコードの先頭で

```
#define mat_elem(mat, i, j) (mat)[(j)][(i)]
```

のように `mat_elem` 関数を定義することである^{*42}。これにより、プログラム中で行列の (i, j) 成分を `mat_elem(A, i, j)` と書けるようになる。

例 2.12.5.

```
#include <stdio.h>
#include <stdlib.h>

#define mat_elem(mat, i, j) (mat)[(j)][(i)]

int main(void) {
    const int m = 3;
    const int n = 4;
    double** matrix = (double**)malloc(sizeof(double*) * n);
    if (matrix == NULL) {
        printf("Can't allocate memory.\n");
        exit(1);
    }
    matrix[0] = (double*)malloc(sizeof(double) * m * n);
    if (matrix[0] == NULL) {
        printf("Can't allocate memory.\n");
        exit(1);
    }
    for (int i = 1; i < n; ++i) {
        matrix[i] = matrix[i - 1] + m;
    }
    for (int i = 0; i < m; ++i) {
        for (int j = 0; j < n; ++j) {
            mat_elem(matrix, i, j) = (i + 1) * (j + 1);
        }
    }
    for (int i = 0; i < m; ++i) {
        for (int j = 0; j < n; ++j) {
            printf("matrix[%d][%d] = %lf\n", i, j, mat_elem(matrix, i, j));
        }
    }
    free(matrix[0]);
    free(matrix);
    return 0;
}
```

プログラムの前半で、 3×4 ではなく 4×3 の 2 次元配列として行列を作成していることに注意せよ^{*43}。

2.13 segmentation fault が出たら

コンパイルは出来たのに、実行してみたら `segmentation fault` と出てプログラムが強制終了することがある^{*44}。こういった場合の典型的な原因と対処法を説明する。

^{*42} 一般的に C プロセッサのマクロの多用は勧められていないが、背に腹は代えられない。

^{*43} 列優勢 (column-major) の 2 次元配列の確保や解放のための関数、および要素へのアクセス用のマクロ一式をまとめたものが、`cmatrix.h` として <https://github.com/utphys-comp/cmatrix/> で公開されている。

^{*44} “することがある” というのが重要。プログラム自体が全く同じでも異常終了したりしなかったりする。

スタックの枯渇

2.3.1. で説明したとおり、動的でない配列を大量に確保するとクラッシュすることがある。他にも関数の呼び出し過多 (主に再帰) でも同様の現象が起こることがある。

■対処法

動的配列に変更する。再帰を for などのより簡単な処理に置き換える。

範囲外参照

動的かそうでないかに関わらず、小さすぎる/大きすぎる値を [] に入れて配列を参照するとクラッシュすることがある。malloc の容量設定ミス (要素数ではなくバイト数) や argc の確認不備でよく発生する。

■対処法

ソースコードを見直す。(例えば 1 次元配列 a[N] に対して参照可能な領域は a[0], a[1]..., a[N-1] だけである。)

ダングリングポインタの参照

代入を行っていないポインタや NULL ポインタ、すでに解放済みの動的配列にアクセスするとクラッシュすることがある。fopen や malloc でよく発生する。

■対処法

ポインタにアクセスする前に、そのポインタがきちんと有効な対象を指しているかを確認する。NULL チェックを行う。

2.14 外部ヘッダの利用

ここでは、他の誰かが作成してくれたものを如何にして自分のプログラムに組み込むか、ということについて説明する。やや難しい内容であるので、読み飛ばしても構わない。

2.14.1 外部ヘッダの利用

既に何度か出てきているが、.h で終わるファイルをヘッダという。ヘッダの役割の一つとして関数の宣言や定義^{*45}を書くというものがある。今回は先程出てきた cmatrix.h を例として、外部ヘッダに書かれた関数の使用方法を説明する。

<https://github.com/utphys-comp/cmatrix/> から cmatrix.h をダウンロードしてきて、以下の例 2.14.1. と同じディレクトリに配置しよう。

例 2.14.1.

```
#include "cmatrix.h"
#include <stdio.h>

int main(void) {
    const int N = 3;
    double** a = alloc_dmatrix(N, N);
    for (int j = 0; j < N; ++j) {
        for (int i = 0; i < N; ++i) {
            mat_elem(a, i, j) = i + j * N;
        }
    }
}
```

^{*45} 定義を書く場合にはやや特殊な工夫が必要になるので、自作する場合には気をつけよ。

```

    }
}
fprintf_dmatrix(stdout, N, N, a);
free_dmatrix(a);
return 0;
}

```

ここで新しく出てきた`#include "file"`の記法はただ通常の`#include <file>`とヘッダ検索の順序が異なるだけのものである。

コンパイルにはいつもどおりでよい。

```
$ gcc 2.14.1.c -Wall -Wextra
```

■コンパイラにヘッダの場所を教える

先程は.cと同じ場所に.hを置いたので、特に何もせずともコンパイルができた。しかし実際にはもっと込み入った場所に置く必要が出てくる場合もある。そんなときには以下の方法を試してみよう。

明示的に場所を指定する

gcc に `-I` オプションを与えると、指定したディレクトリをヘッダのある場所として検索しに行くようになる。

CPATH を設定する

詳しく説明はしないが、`export` コマンドを使って `CPATH` という環境変数を改変することで gcc がヘッダを探索しに行く場所を増やすことができる。変更はターミナルからログアウトするまでずっと有効である。

シンボリックリンクを貼る

`ln -s` を使ってシンボリックリンクを作ると、全く別の場所にあるヘッダがあたかも.cの隣に置いてあるかのように偽装することができる。参照先のファイルを変更すると手元にあるリンクも自動でそれに追従するので、コピーするよりもスマートである。

2.14.2 外部ライブラリの利用

次は同じようなことをライブラリに対してやってみよう。例として何度か出てきている LAPACK を使用する。標準的な環境では特に何もせずとも使えるはずであるが、以下のサンプルが動かないようなら自分の環境に合わせた方法でインストールすること。

以下の例 2.14.2. を `cmatrix.h` と同じディレクトリに配置しよう。

例 2.14.2.

```

#include "cmatrix.h"
#include <assert.h>
#include <stdio.h>

extern void dgesv_(const int*, const int*, double*, const int*, int*, double*,
                  const int*, int*);

int main(void) {
    const int N = 3;
    const int NRHS = 1;
    const int LDA = N;
    const int LDB = N;
    double** a = alloc_dmatrix(N, N);

```

```

double* b = alloc_dvector(N);
int* ipiv = alloc_ivector(N);
for (int i = 0; i < N; ++i) {
    mat_elem(a, i, i) = 1;
    if (i + 1 != N) {
        mat_elem(a, i, i + 1) = -1;
    }
}
b[N - 1] = 1;
int info;
dgesv_(&N, &NRHS, mat_ptr(a), &LDA, vec_ptr(ipiv), vec_ptr(b), &LDB, &info);
assert(info == 0);
fprintf_dvector(stdout, N, b);
free_dmatrix(a);
free_dvector(b);
free_ivector(ipiv);
return 0;
}

```

今回は `-llapack -lblas` のオプションが必要になる。LAPACK が BLAS に依存している都合上、-1 の順番を変えてはならない。

```
$ gcc 2.14.2.c -llapack -lblas -Wall -Wextra
```

ここで何をしているのかについては説明しないが、<https://netlib.org/lapack/explore-html/index.html> を使えば調べられる。最初に書いてある `dgesv_` の宣言部分は 2.7.6. でやっていることと本質的に同じであるが、LAPACK のソースを見て引数の型を調べるのは大変なので、そのときにもこのページが役に立つだろう^{*46}。

■コンパイラにライブラリの場所を教える

今回何も工夫をせずともコンパイルができたのは、LAPACK がシステムの奥深くに配置されており、その場所が `gcc` の自動探索対象箇所であったからである^{*47}。よってこれほどすんなりとは行かないことも十分ありえる。そんなときには以下の方法を試してみよう。

明示的に場所を指定する

`gcc` に `-L` オプションを与えると、指定したディレクトリをライブラリのある場所として探索しに行くようになる。

LD_LIBRARY_PATH を設定する

詳しく説明はしないが、`export` コマンドを使って `LD_LIBRARY_PATH` という環境変数を改変することで `gcc` がライブラリを探索しに行く場所を増やすことができる。変更はターミナルからログアウトするまでずっと有効である。

2.15 外部ツールの利用

C 言語でのプログラミングにおいて有用な外部ツールをいくつか紹介する。あくまで“紹介”に留めるので、使い方は各自調べること。

make

コンパイル用コマンドを管理するツール。

^{*46} 安全性のため Fortran における入力専用変数にはすべて `const` をつけてみたが、難しかったら取り除いても構わない。

^{*47} インストールの方法によってはそういった場所に置いてくれないこともある。

cmake

コンパイルの設定、依存関係、テストなどを管理するツール。

git

変更履歴を管理するツール。

gdb

デバッガ。指定した行で処理を止めたり、変数の中身を覗いたり書き換えたりできる。

valgrind

メモリリークや範囲外参照を検出できるツール。

プロファイラ

プログラムの処理に時間を要している部分を解析するツール。gprof や vtune などが有名。

フォーマッタ

インデントやスペース等を全自動で揃えてくれるツール。C 用としては clang-format が有名。

2.16 練習問題の解答例

以下に練習問題の解答例を示す。さまざまなプログラムの書き方があるので、これらの解答例にこだわらないこと。全く分からない場合に参考にする程度が望ましい。

練習 2.2.1.

```
#include <stdio.h>

int main(void) {
    /* ax+b=0 */
    const double a = 4.0;
    const double b = 1.0;
    printf("ax+b=0\n");
    printf("  a = %lf\n", a);
    printf("  b = %lf\n\n", b);
    if (a == 0.0) {
        if (b == 0.0) {
            printf("  x = all\n");
        } else {
            printf("  x = nothing\n");
        }
    } else {
        printf("  x = %lf\n", -b / a);
    }
    return 0;
}
```

練習 2.2.2.

```
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    /* calculate "n!" */
    const int n = 10;
    if (n < 0) {
        printf("Can't calculate n!.\n");
        printf("n = %d\n", n);
        exit(1);
    }
}
```

```

    }
    int ans = 1;
    for (int i = 1; i <= n; ++i) {
        ans *= i;
    }
    printf("%d! = %d\n", n, ans);
    return 0;
}

```

練習 2.2.3.

```

#include <stdio.h>

int main(void) {
    /* n madeno sosuu. */
    const int n = 100;
    printf("%d madeno sosuu = ", n);
    int ans = 2;
    while (ans <= n) {
        int flag = 1;
        for (int i = 2; i < ans; ++i) {
            if (ans % i == 0) {
                flag = 0;
                break;
            }
        }
        if (flag == 1) {
            printf("%d, ", ans);
        }
        ++ans;
    }
    printf("\n");
    return 0;
}

```

練習 2.3.1.

```

#include <stdio.h>

int main(void) {
    int a[5] = { 1, 2, 3, 4, 5 };
    for (int i = 0; i < 5; ++i) {
        printf("Start: a[%d] = %d\n", i, a[i]);
    }
    int b[5];
    for (int i = 0; i < 5; ++i) {
        b[i] = a[i];
        a[i] = 0;
        printf("End   : a[%d] = %d, b[%d] = %d\n", i, a[i], i, b[i]);
    }
    return 0;
}

```

練習 2.3.2.


```

#include <stdio.h>

void seki(double a[2][2], double b[2][2], double c[2][2]) {
    c[0][0] = a[0][0] * b[0][0] + a[0][1] * b[1][0];
    c[0][1] = a[0][0] * b[0][1] + a[0][1] * b[1][1];
    c[1][0] = a[1][0] * b[0][0] + a[1][1] * b[1][0];
    c[1][1] = a[1][0] * b[0][1] + a[1][1] * b[1][1];
}

int main(void) {
    double a[2][2] = { { 1.0, 2.0 }, { 3.0, 4.0 } };
    double b[2][2] = { { -1.0, -2.0 }, { -3.0, -4.0 } };
    for (int i = 0; i < 2; ++i) {
        for (int j = 0; j < 2; ++j) {
            printf("a[%d][%d] = %lf, b[%d][%d] = %lf\n", i, j, a[i][j], i, j,
                b[i][j]);
        }
    }
    double c[2][2];
    seki(a, b, c);
    for (int i = 0; i < 2; ++i) {
        for (int j = 0; j < 2; ++j) {
            printf("(a x b)[%d][%d] = %lf\n", i, j, c[i][j]);
        }
    }
    return 0;
}

```

練習 2.4.1.

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(void) {
    /* calculate "n!" */
    int n;
    printf("Input(int): ");
    scanf("%d", &n);
    if (n < 0) {
        printf("Can't calculate n!.\n");
        printf("n = %d\n", n);
        exit(1);
    }
    int ans = 1;
    for (int i = 1; i <= n; ++i) {
        ans *= i;
    }
    printf("%d! = %d\n", n, ans);
    return 0;
}

```

練習 2.5.1.

```

#include <stdio.h>

```

```

int main(void) {
    int array[10];
    int* p;
    for (int i = 0; i < 10; ++i) {
        array[i] = i * i;
    }
    for (int i = 0; i < 10; ++i) {
        printf("array[%d] = %d\n", i, *(array + i));
    }
    for (int i = 0; i < 10; ++i) {
        p = &(array[i]);
        printf("array[%d] = %d\n", i, *p);
    }
    return 0;
}

```

練習 2.8.1.

```

#include <stdio.h>

struct date {
    unsigned day, month;
    unsigned days;
};

int main(void) {
    const unsigned nMonth[12] = { 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30,
                                   31 };
    struct date inputDay;
    printf("Input(month & day) :\n");
    printf("          month :");
    scanf("%d", &(inputDay.month));
    printf("          day  :");
    scanf("%d", &(inputDay.day));
    inputDay.days = 0;
    for (int i = 1; i < inputDay.month; ++i) {
        inputDay.days += nMonth[i - 1];
    }
    inputDay.days += inputDay.day;
    printf("%d/%d = %d days from 1/1\n", inputDay.month, inputDay.day,
        inputDay.days);
    return 0;
}

```

第3章

L^AT_EX 入門

L^AT_EX は L. Lamport の開発した文書清書ソフトウェアである。もともとは、D. Knuth の開発した T_EX と呼ばれるソフトウェアで、これを使いやすく拡張したものである。L^AT_EX はワードプロセッサのようなソフトウェアではない。画面を見ながら文書を整形しその見たままの姿が印刷結果となるのがワードプロセッサである。例えば代表的なワードプロセッサである Microsoft Office Word では、段落や 2 段組、箇条書などの編集を画面を見ながら行えて、そして画面のイメージがほぼそのまま印刷される。一方、L^AT_EX では見たままが印刷結果となるわけではないが、ワードプロセッサに匹敵する強力な清書機能を持っていて、とくに数式の出力に際して威力を発揮する。

3.1 L^AT_EX の実行

では、さっそく例を見てみよう。適当なエディタで

例 3.1.1.

```
\documentclass[uplatex,dvipdfmx]{jsarticle}
\begin{document}
\(\alpha\) のまわりのテイラー展開
\begin{equation}
f(x) = \sum_{n=0}^{\infty} \frac{f^{(n)}(a)}{n!} (x-a)^n
\end{equation}
\end{document}
```

と入力してみよう。このファイルを `ex.tex` という名前で保存する。このファイルを L^AT_EX で処理するには、`uplatex` コマンド^{*1}と `dvipdfmx` コマンドを実行すればよい^{*2}。

```
$ uplatex ex.tex
$ dvipdfmx ex.dvi
```

1 行目の `platex` コマンドで DVI ファイル (`ex.dvi`) が生成される。DVI は **d**evice-**i**ndependent **f**ile **f**ormat の略で、最終的な出力形式 (今の例では PDF) に依存しない出力形式である。`dvipdfmx` コマンドは、この DVI ファイルから PDF 形式のファイルを生成する。これらのコマンドの実行後、`ex.pdf` というファイルができていれば成功である。では処理結果を見てみよう。macOS で作業している場合には、

```
$ open ex.pdf
```

^{*1} 日本語版 L^AT_EX。英語のみを含む文書を処理する場合には、`latex` コマンドを使えばよい。

^{*2} Overleaf で日本語の文書を作成する際に必要となる設定については、<https://utphys-comp.github.io/latex> を参照のこと。

Linux で作業している場合には、

```
$ evince ex.pdf
```

で PDF ファイルを開くことができる。結果は以下のようにになっているはずである。

結果 3.1.1.

a のまわりのテイラー展開

$$f(x) = \sum_{n=0}^{\infty} \frac{f^{(n)}(a)}{n!} (x-a)^n \quad (1)$$

■この章のまとめ

- L^AT_EX ファイルの拡張子は `.tex` である。
- L^AT_EX を使うには、`uplatex` コマンドを実行する。
- `uplatex` コマンドを実行すると、DVI ファイル (拡張子は `.dvi`) が生成される。
- DVI ファイルを PDF 形式に変換するには、`dvipdfmx` コマンドを実行する。
- PDF ファイルを見るには、“open ファイル名” (macOS の場合)、あるいは “`evince ファイル名`” (Linux の場合) を実行する。

3.2 L^AT_EX に関する全体的なこと

すでに例 3.1.1. で見たとおり、L^AT_EX のソースファイルには、本文以外の情報も書き込まれている。具体的には、

例 3.2.1.

```
\documentclass[uplatex,dvipdfmx]{jsarticle}
\begin{document}

% ここに本文を書く

\end{document}
```

のところがかならず書かなくてはならないところである。特に `\begin{document}` より前の部分をプリアンブルと呼ぶ。以下の例題では、このかならず書かなくてはいけない部分を省略してあるので注意すること。

さて、例 3.2.1. にはパーセント記号 (%) で始まる行が書かれている。L^AT_EX では、% で始まる行はコメントとみなされる。行の途中で % が現われたときは、そこから行の最後までがコメントになる。コメントは L^AT_EX の動作には影響を与えない。

文章はソースファイルの中にそのまま書けばよい。ソースファイルの途中で改行しても清書の結果には影響を与えないので、見やすいように適宜改行を挟むとよい^{*3}。また、空行を挟むとそこで改段落される。

例 3.2.2.

スーパーカミオカンデ実験の説明

スーパーカミオカンデは、世界最大の水チェレンコフ

^{*3} L^AT_EX にも `latexindent` というフォーマッタがあり、勝手に見やすい形に直してくれる。今回出てくる例はすべてこれで整形してある。

宇宙素粒子観測装置です。1991 年に建設が始まり、
5 年間にわたる建設期間を経たのち、1996 年 4 月より
観測を開始しました。

この出力は、

結果 3.2.2.

スーパーカミオカンデ実験の説明

スーパーカミオカンデは、世界最大の水チェレンコフ宇宙素粒子観測装置です。1991 年に建設が始まり、5 年間にわたる建設期間を経たのち、1996 年 4 月より観測を開始しました。

となる。出力は、もっとも美しく清書されるように、 $\text{\LaTeX}$ が自動的に改行を行ってくれる。逆にどうしても自分で改行させたいときは、`\\`をソースファイル中に書き込めばよい。

例 3.2.3.

東京大学物性研究所 \\ \\
秋葉原からつくばエクスプレスに乗り、
柏の葉キャンパス前で下車します。\\
運賃は 670 円です。\\
所要時間はおおよそ 30 分です。

結果は次のとおりである。

結果 3.2.3.

東京大学物性研究所

秋葉原からつくばエクスプレスに乗り、柏の葉キャンパス前で下車します。
運賃は 670 円です。
所要時間はおおよそ 30 分です。

空行を挟む改段落と異なりあくまで改行に過ぎないので、次の文がインデントされないことに注意せよ。

■この節のまとめ

- ソースファイルには必ず書かなくてはいけない部分がある。
- `%` から行の最後まではコメントである。
- 文章はソースファイル中に直接書き込む。その際、改行は無視される。
- 空行を挟むとそこで改段落される。
- 強制改行をするには `\\` を使う。

3.3 フォント

印刷される文字を強調するために、文字を太くしたり (ボールド体)、ななめにしたり (斜体) することがある。印刷される文字の形状のことをフォントといい、その形状を変更することをフォントを変更するといいう。ちなみに、通常のフォントは立体と呼ばれる。

ボールド体はとくに強調したい言葉に使う。文章中でフォントを変更するには、

例 3.3.1.

東京駅から東京大学に向かうには、
`\textbf{地下鉄丸ノ内線}`を使うのが便利です。

```
\textbf{池袋}ゆきに乗る\textbf{本郷三丁目}駅で下車します。
```

のように`\textbf{文字}`命令を使う。結果は、

結果 3.3.1.

東京駅から東京大学に向かうには、**地下鉄丸ノ内線** を使うのが便利です。**池袋**ゆきに乗る**本郷三丁目**駅で下車します。

となる。斜体は、変数などのパラメータを示したり、英単語を強調するのに用いる。

例 3.3.2.

```
\textbf{ls} コマンドに \textit{file} 引数を渡すと  
その \textit{file} に関する情報を表示します。
```

結果 3.3.2.

`ls` コマンドに *file* 引数を渡すとその *file* に関する情報を表示します。

タイプライタ体は計算機への入力、計算機からの出力、プログラムなどを示すのに使う。

例 3.3.3.

標準ライブラリ関数には、`\texttt{sin()}` や `\texttt{cos()}`、`\texttt{tan()}` といった三角関数も用意されています。

結果 3.3.3.

標準ライブラリ関数には `sin()` や `cos()`、`tan()` といった三角関数も用意されています。

■この節のまとめ

- フォントの種類を変更するには `\textbf{文字}`、`\textit{文字}`、`\texttt{文字}` などを使う。

3.4 論文のスタイル

L^AT_EX には論文を清書するのに便利な機能が備わっている。

3.4.1 表題

通常、論文の表題には、論文の題名、著者、著者の所属、提出日時、論文の要旨などを書く。

例 3.4.1.

```
\documentclass[uplatex,dvipdfmx]{jsarticle}
% 表題、著者、所属、提出日時
\title{各種プログラミング言語に関する考察}
\author{物理太郎\東京大学大学院理学系研究科物理学専攻}
\date{2019 年 4 月 8 日}
\begin{document}

% 表題の表示
\maketitle
```

%% ここに論文の本文を書く %%

\end{document}

レポート程度の簡単なものならば論文の要旨は不要であろう。結果は次のとおり。

結果 3.4.1.

各種プログラミング言語に関する考察

物理太郎

東京大学大学院理学系研究科物理学専攻

2019 年 4 月 8 日

3.4.2 論文の構成

通常、論文は内容にしたがって章分けされる。以下で $\text{\LaTeX}$ で章を分ける例を示す。

例 3.4.2.

```
\section{C 言語}
C 言語は、
Brain Kernighan と Dennis Ritchie によって開発された
プログラミング言語である。
簡潔性と柔軟性の両方を備えたこの言語は、
さまざまな分野のソフトウェア開発に広く利用されている。
われわれが普段利用している UNIX オペレーティングシステムも
この C 言語を用いて記述されたものである。

\dots

\subsection{C 言語の歴史}
C 言語はベル研究所の Ritchie によって 1972 年頃開発された。
C 言語は Algol と呼ばれるプログラミング言語の流れを受け継ぐ言語で、
直接の母体は B 言語と呼ばれる言語である。

\dots

\subsection{C 言語の特徴}
この節では C 言語の特徴について述べよう。

\subsubsection{構造化プログラミング言語}
C 言語にはデータのまとまりを表す構造体と呼ばれるデータ型がある。
データを集合として扱う構造データ型は FORTRAN にはなかった。

\dots

\subsubsection{機械指向型プログラミング言語}
C 言語の一つの命令は FORTRAN などに比べると簡潔であり
計算機に与えられる命令に近い。
これはプログラミング言語自体による副作用なしに
計算機を思い通りに直接操作できることを意味する。

\dots

\section{FORTRAN}
FORTRAN は FORMula TRANslator の名が示すとおり、
```

科学技術計算の能力に長けたプログラミング言語である。

`\dots`

このように、章や節にタイトルを付けるには、`\chapter{}`、`\section{}`、`\subsection{}`、`\subsubsection{}`を使う。タイトルを`{}`の中に書き、そのあとには本文となる文章を続ける。章や節の番号は $\text{\LaTeX}$ が自動的に振ってくれる。結果は次のようになる。

結果 3.4.2.

1 C 言語

C 言語は、Brain Kernighan と Dennis Ritchie によって開発されたプログラミング言語である。簡潔性と柔軟性の両方を備えたこの言語は、さまざまな分野のソフトウェア開発に広く利用されている。われわれが普段利用している UNIX オペレーティングシステムもこの C 言語を用いて記述されたものである。

...

1.1 C 言語の歴史

C 言語はベル研究所の Ritchie によって 1972 年頃開発された。C 言語は Algol と呼ばれるプログラミング言語の流れを受け継ぐ言語で、直接の母体は B 言語と呼ばれる言語である。

...

1.2 C 言語の特徴

この節では C 言語の特徴について述べよう。

1.2.1 構造化プログラミング言語

C 言語にはデータのまとまりを表す構造体と呼ばれるデータ型がある。データを集合として扱う構造データ型は FORTRAN にはなかった。

...

1.2.2 機械指向型プログラミング言語

C 言語の一つの命令は FORTRAN などと比べると簡潔であり計算機に与えられる命令に近い。これはプログラミング言語自体による副作用なしに計算機を思い通りに直接操作できることを意味する。

...

2 FORTRAN

FORTRAN は FORMula TRANslator の名が示すとおり、科学技術計算の能力に長けたプログラミング言語である。

...

■この節のまとめ

- 論文の表題は、`\title{}`、`\author{}`、`\date{}`などで指定する。
- 表題を出力するには、`\maketitle`を使う。
- 章や節のタイトルを付けるには、`\chapter{}`、`\section{}`、`\subsection{}`、`\subsubsection{}`などを用いる。

3.5 箇条書き

論文の中では箇条書きを使うことは避けたほうがよいといわれている。特に、何かの処理の手順を説明するときは、箇条書きにするよりは言葉を使って説明するべきである。しかし、場合によっては箇条書きの方が説明が伝わりやすいこともある。単に項目を並列に並べたいときは箇条書きでもよいであろう。この節では箇条書きの方法を説明する。箇条書きにしたいときは次のように書く。

例 3.5.1.

午後の紅茶（ミルクティー）の原材料名：

```
\begin{itemize}
  \item 牛乳
  \item 砂糖
  \item 加糖練乳
  \item 紅茶
  \item 香料
  \item 乳化剤
  \item ビタミン C
\end{itemize}
```

このように、文章を`\begin{itemize}`と`\end{itemize}`とで囲むと、その領域は箇条書き環境になる。箇条書き環境の中では、`\item`というコマンドを使うと新しい項目を始めることができる。

結果 3.5.1.

午後の紅茶（ミルクティー）の原材料名：

- 牛乳
- 砂糖
- 加糖練乳
- 紅茶
- 香料
- 乳化剤
- ビタミン C

また、`itemize`の代わりに`enumerate`を使うと、各項目のラベルが1, 2, 3, …のような番号となる。

例 3.5.2.

この講義の評価について該当するものに丸を付けて下さい。

```
\begin{enumerate}
  \item 大変良かった。
  \item まあまあ良かった。
  \item 普通だった。
  \item あまり良くなかった。
  \item かなり良くなかった。
\end{enumerate}
```

結果 3.5.2.

この講義の評価について該当するものに丸を付けて下さい。

1. 大変良かった。
2. まあまあ良かった。
3. 普通だった。
4. あまり良くなかった。
5. かなり良くなかった。

■この節のまとめ

- `\begin{itemize}` と `\end{itemize}` で囲まれた部分は箇条書き環境になる。
- `\begin{enumerate}` と `\end{enumerate}` で囲まれた部分は番号付き箇条書き環境になる。
- 箇条書きの項目を始めるには`\item`を使う。

3.6 表の作成

表を作成するには次に示す雛型を使えばよい。

例 3.6.1.

```
\begin{table}
\centering
\caption{月刊誌の出版社と発売日}
\begin{tabular}{|l|l|l|}
\hline
雑誌名           & 出版社       & 発売日  \\
\hline \hline
トランジスタ技術 & CQ 出版      & 10 日   \\
\hline
UNIX Magazine     & アスキー     & 19 日   \\
\hline
SD ソフトウェアデザイン & 技術評論社  & 19 日   \\
\hline
アフタヌーン      & 講談社       & 25 日   \\
\hline
\end{tabular}
\end{table}
```

結果 3.6.1.

表 1: 月刊誌の出版社と発売日

雑誌名	出版社	発売日
トランジスタ技術	CQ 出版	10 日
UNIX Magazine	アスキー	19 日
SD ソフトウェアデザイン	技術評論社	19 日
アフタヌーン	講談社	25 日

`\hline` と書いたところには、横線が引かれる。`\hline \hline` と二つ続けて書くと二重線になる。表の各行は、それぞれの要素の間を`&`で区切り、最後に `\\` で改行して行が終わったことを $\text{\LaTeX}$ に伝える。

この表は横の 1 行に三つの要素を持つ表である。各要素は表の 1 マスに左詰めで書かれている。このことは

```
\begin{tabular}{|l|l|l|}
```

の`{|l|l|l|}`によって表現されている。1 は左詰めを意味し、それを三つ並べることで、1 行に三つの要素が入ることを表現している。左詰めの代わりに中央揃えがよければ `c` を、右詰めがよければ `r` を指定する。

それぞれの 1 の両隣の縦棒は、要素と要素の間を縦線で区切って印刷してほしいということを $\text{\LaTeX}$ に伝えている。この縦棒を取ってしまうと、要素間の縦線が引かれなくなる。

表の表題は

```
\caption{月刊誌の出版社と発売日}
```

のように `\caption` の後の `{}` の中に書く。最後にもう一つ例を挙げておこう。

例 3.6.2.

```
\begin{table}
\centering
\caption{代表的な粒子の質量}
\begin{tabular}{|c|r|}
\hline
粒子 & 質量 (MeV/(c^2)) \\
\hline
\hline
(e ~ \pm) & 0.5110 \\
\hline
(\mu ~ \pm) & 105.7 \\
\hline
(\pi ~ \pm) & 139.6 \\
\hline
(\pi ~ 0) & 135.0 \\
\hline
(J/\psi) & 3097 \\
\hline
(B ~ 0) & 5280 \\
\hline
\end{tabular}
\end{table}
```

結果 3.6.2.

表 1: 代表的な粒子の質量

粒子	質量 (MeV/ c^2)
$e^\pm$	0.5110
$\mu^\pm$	105.7
$\pi^\pm$	139.6
π^0	135.0
J/ψ	3097
B^0	5280

3.7 数式

数式は、例 3.1.1. で見たとおり、

例 3.7.1.

```
\begin{equation}
%%% ここに数式を書きます %%%
\end{equation}
```

というスタイルで記述する。`\begin{equation}`と`\end{equation}`とで囲まれた範囲では、数式環境の中のみで有効な様々なコマンドが使える。また、数式の番号は自動的に振られる。

結果 3.1.1. で見たとおり、`\begin{equation}`と`\end{equation}`とで囲まれた領域の数式は、改まった行の中央に表示される。しかし、場合によっては

結果 3.7.2.

x の関数 $y = ax^2 + bx + c$ のうち $a \neq 0$ であるものを x の 2 次関数といいます。

のように文章中にインラインで数式を書きたいこともあるだろう。そういう場合は、文章中に`\(と\)` で囲った数式を書けばよい。結果 3.7.2. を与えるソースは、以下のとおりである。

例 3.7.2.
`\(x\) の関数\(\(y = ax ^ 2 + bx + c\) のうち`
`\(a \neq 0\) であるものを\(\(x\) の 2 次関数といいます。`

以降の例題中ではスペースの都合上、二つ以上の数式を並べて書いてある場合があるが、実際には一つずつ`\begin{equation}`と`\end{equation}`とで囲むか、`\(と\)` とで囲まなくてはならない。

3.7.1 添え字

上付き添え字は`^`で、下付き添え字は`_`で表現する。

例 3.7.3.
`y = a x ^ 2 + b x + c`
`S_n = a_1 + a_2 + a_3 + \cdots + a_n`

結果 3.7.3.
$$y = ax^2 + bx + c$$
$$S_n = a_1 + a_2 + a_3 + \cdots + a_n$$

2 文字以上からなる項を添え字にしたいときは、`{ }`で添え字を囲む。

例 3.7.4.
`Tr\,A = A_{11} + A_{22} + A_{33} + \cdots + A_{ii} + \cdots + A_{nn}`

結果 3.7.4.
$$Tr A = A_{11} + A_{22} + A_{33} + \cdots + A_{ii} + \cdots + A_{nn}$$

添え字に限らず、`LaTeX` の数式コマンドを 2 文字以上の項に作用させるときはかならず項を`{ }`で囲む必要がある。逆に 1 文字だけの項を`{ }`で囲んでも特に問題はない。なお、例 3.7.4. の中で `\,,'` というコマンドが使われているが、これはスペースの微調整をしたいときに使用する。

3.7.2 分数

分数は`\frac{分子}{分母}`のように表現する。

例 3.7.5.
`1 + \frac{1}{2} = \frac{3}{2}`

$$1 + \frac{1}{1+\frac{1}{2}} = \frac{5}{3}$$

結果 3.7.5.

$$1 + \frac{1}{2} = \frac{3}{2}$$

$$1 + \frac{1}{1 + \frac{1}{2}} = \frac{5}{3}$$

3.7.3 関数

数学環境では文字はすべて変数と見なされるため、`\cos x` という入力は $\cos x$ と斜体で印刷される。これではあたかも c 、 o 、 s 、 x の積のようである。数学関数は正しくは立体で表示されなくてはならない。立体で印刷するためには、`\cos x` と入力する。このように数学関数の多くはバックスラッシュ (`\`) に関数名をつなげると表現できる。

例 3.7.6.

$$\frac{d}{dx} \sin x = \cos x$$

結果 3.7.6.

$$\frac{d}{dx} \sin x = \cos x$$

表 3.1 に主な数学関数を示す。

表 3.1 主な数学関数

<code>\cos</code>	<code>\sin</code>	<code>\tan</code>
<code>\cosh</code>	<code>\sinh</code>	<code>\tanh</code>
<code>\log</code>	<code>\ln</code>	<code>\exp</code>
<code>\lim</code>		

3.7.4 フォント

前節でも触れたが、 e は立体で印刷したい文字も斜体で印刷してしまうことがある。数式中でフォントを強制的に変更するときは前に説明した方法は使わない。数式中で立体の文字を印刷するには、`\mathrm{文字}` と書く。

例 3.7.7.

$$\frac{d}{dx} \mathrm{e}^x = \mathrm{e}^x$$

これにより、立体で表示されるべきところは正しく立体で印刷される。

結果 3.7.7.

$$\frac{d}{dx} \mathrm{e}^x = \mathrm{e}^x$$

3.7.5 積分・和・積

積分、和、積は以下のように表現する。

例 3.7.8.

```
\int _1 ^2 \frac{1}{x^2} \mathrm{d}x = \frac{1}{2}
\sum _{k=1} ^n k = \frac{n(n+1)}{2}
n! = \prod _{k=1} ^n k
```

結果 3.7.8.

$$\int_1^2 \frac{1}{x^2} dx = \frac{1}{2}$$

$$\sum_{k=1}^n k = \frac{n(n+1)}{2}$$

$$n! = \prod_{k=1}^n k$$

3.7.6 根号・ベクトル・上線・チルダ・ハット・時間微分

根号、ベクトル、上線の記号は以下のように表現する。

例 3.7.9.

```
\sqrt{x^2 + 2x + 1} = |x+1|
\vec{x} + \vec{y} = \vec{z}
\overrightarrow{OA} + \overrightarrow{AP} = \overrightarrow{OP}
\overline{x + \mathrm{i}y} = x - \mathrm{i}y
```

結果 3.7.9.

$$\sqrt{x^2 + 2x + 1} = |x + 1|$$

$$\vec{x} + \vec{y} = \vec{z}$$

$$\overrightarrow{OA} + \overrightarrow{AP} = \overrightarrow{OP}$$

$$\overline{x + \mathrm{i}y} = x - \mathrm{i}y$$

同様にチルダ、ハット（演算子）、時間微分などは以下のように表現する。

例 3.7.10.

```
\tilde{x}
\hat{r}
\dot{x}
```

結果 3.7.10.

$$\tilde{x}$$

$$\hat{r}$$

$$\dot{x}$$

3.7.7 ギリシャ文字

ギリシャ文字は文字を英語表記で綴り、前にバックスラッシュを付けると表示される。 α は `\alpha` と入力する。ギリシャ文字の大文字は、英語の綴りの最初の 1 文字を大文字にすると表示できる。`\Psi` と書けば Ψ が印刷される。

表 3.2 ギリシャ文字一覧

コマンド	小文字	大文字	コマンド	小文字	大文字	コマンド	小文字	大文字
<code>\alpha</code>	α	—	<code>\iota</code>	ι	—	<code>\rho</code>	ρ	—
<code>\beta</code>	β	—	<code>\kappa</code>	κ	—	<code>\sigma</code>	σ	Σ
<code>\gamma</code>	γ	Γ	<code>\lambda</code>	λ	Λ	<code>\tau</code>	τ	—
<code>\delta</code>	δ	Δ	<code>\mu</code>	μ	—	<code>\upsilon</code>	υ	Υ
<code>\epsilon</code>	ϵ	—	<code>\nu</code>	ν	—	<code>\phi</code>	ϕ	Φ
<code>\zeta</code>	ζ	—	<code>\xi</code>	ξ	Ξ	<code>\chi</code>	χ	—
<code>\eta</code>	η	—	<code>o</code>	o	O	<code>\psi</code>	ψ	Ψ
<code>\theta</code>	θ	Θ	<code>\pi</code>	π	Π	<code>\omega</code>	ω	Ω

— となっている箇所は、通常のアルファベットの太文字で代用する。

3.7.8 行列

行列を書く方法は表の作成と似ている。

例 3.7.11.

```
\sigma_3 = (
\begin{array}{cc}
1 & 0 \\
0 & -1
\end{array} )
```

表のところで説明されている書き方のうち、`\begin{tabular}` から `\end{tabular}` までをまねして、`tabular` を `array` に書き換える。行列の丸カッコは自動的に印刷されないので自分で書く^{*4}。

結果 3.7.11.

$$\sigma_3 = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$$

しかし、これではカッコの大きさがあまり美しくない。カッコを括られている内容の高さに合わせて大きくす

^{*4} `amsmath` パッケージにある `pmatrix` 環境を使うと行列をより自然な形で記述できる。`amsmath` パッケージの利用方法については 3.8.1 を見よ。

るには、`\left(` と `\right)` を使う。

例 3.7.12.

```
\phi (x) = \sqrt{\frac{1}{2}} \left(
\begin{array}{c}
0 \\
v + h(x)
\end{array} \right)
```

結果 3.7.12.

$$\phi(x) = \sqrt{\frac{1}{2}} \left(\begin{array}{c} 0 \\ v + h(x) \end{array} \right) \quad (1)$$

`\left(` と `\right)` の組み合わせは、中に入るものが行列でなくても使える。また、丸カッコ以外にも使える。

例 3.7.13.

```
J_{\mu}^{\sim\{NC\}}(\nu) =
\frac{1}{2}
\left[
\overline{u}_{\nu}\gamma_{\mu} \frac{1}{2} (1-\gamma^5)u_{\nu}
\right]
```

結果 3.7.13.

$$J_{\mu}^{NC}(\nu) = \frac{1}{2} \left[\bar{u}_{\nu} \gamma_{\mu} \frac{1}{2} (1 - \gamma^5) u_{\nu} \right] \quad (1)$$

このように角カッコなどにも使える。

3.7.9 特殊記号

最後に $\text{\LaTeX}$ で使える主な特殊記号を示しておく。

表 3.3 主な特殊記号

コマンド	印刷結果
<code>\ne</code>	$\neq$
<code>\le</code>	$\leq$
<code>\ge</code>	$\geq$
<code>\equiv</code>	$\equiv$
<code>\pm</code>	$\pm$
<code>\times</code>	$\times$
<code>\infty</code>	∞
<code>\bullet</code>	$\bullet$
<code>\prime</code>	$\prime$
<code>\dagger</code>	$\dagger$
<code>\partial</code>	∂
<code>\to</code>	$\rightarrow$
<code>\nabla</code>	∇
<code>\triangle</code>	$\triangle$

■この節のまとめ

- 数式を書くには、`\begin{equation}`と`\end{equation}`で囲む。
- 文章中で数式を書くには `\(` と `\)` で囲む。
- 添え字、分数、関数、積分記号などについては、それぞれにコマンドが用意されている。

3.8 図の貼り込み

LaTeX では、Gnuplot (1.4 節) などで生成した図のファイル (PDF など) を文章中に貼り込むことが可能である。

3.8.1 パッケージについて

画像の添付は LaTeX の非標準機能である `graphicx` パッケージを用いて行う。パッケージを使用する際には、以下のようにしてプリアンブル部分に`\usepackage{package name}`の文言を追加すればよい。

例 3.8.1.

```
\documentclass[uplatex,dvipdfmx]{jsarticle}
\usepackage{graphicx}
\begin{document} ...
```

以後断りなく `graphicx` の機能を使うので注意すること。

`graphicx` 以外にもよく使われるパッケージとして、様々な機能を追加し実質的に公式機能と見なされている `amsmath`、後に出てくる `here`、物理でよく使う表現を簡単に入力できる `physics`、ハイパーリンク機能を提供する `hyperref` などがある。これらのパッケージは `texlive-full` をインストールすれば特に何も設定をせずとも使えるはずだ。

3.8.2 PDF ファイルの貼り込み

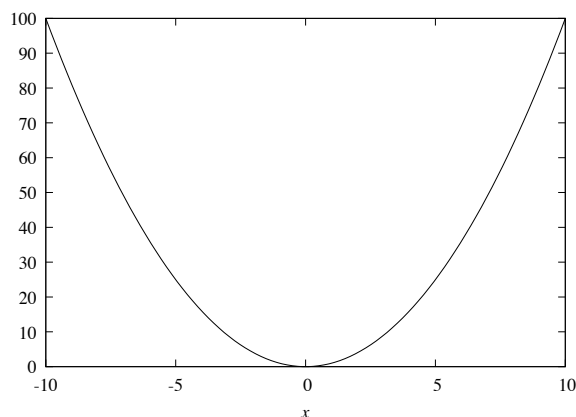
図を貼り込むには以下の雛型を用いる。

例 3.8.2.

```

\begin{figure}
  \centering
  \includegraphics[width=8cm]{figure.pdf}
  \caption{\(y = x^2\) のグラフ}
\end{figure}

```

結果 3.8.2.図 1: $y = x^2$ のグラフ

注意すべきところは

```
\includegraphics[width=8cm]{figure.pdf}
```

という部分である。ファイル名は{}の中で指定する。また `width=8cm` という記述により図の横幅を指定している。図の表題も表のときと同じく `\caption` コマンドを使う。

LaTeX は図表を自動的に適切な位置に配置してくれるが、`\begin{figure}` の後に `[h]` を付けると、その場に配置することができる。それでもうまくいかない場合には、`here` パッケージを使い、**例題 3.8.3.** のように `\begin{figure}` の後に `[H]` を付けることにより、強制的に指定の位置に配置することも出来る。

例 3.8.3.

```

\documentclass[uplatex,dvipdfmx]{jsarticle}
\usepackage{graphicx}
\usepackage{here}

\begin{document}
\begin{figure}[H]
  \centering
  \includegraphics{figure.pdf}
  \caption{\(y = x^2\) のグラフ}
\end{figure}
\end{document}

```

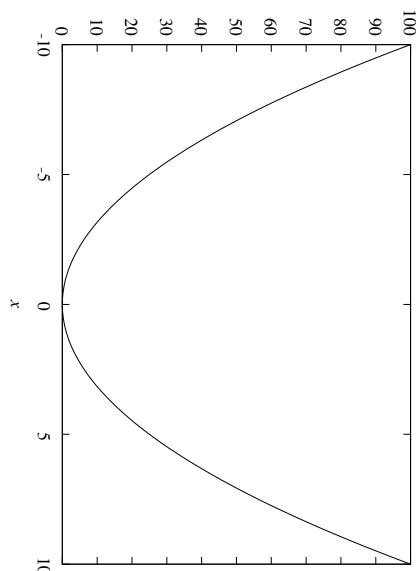
3.8.3 図の回転

貼り込みたいファイル中の図が 90 度回転してしまっていることがある。LaTeX では図を貼り込むときに回転させることができる。

例 3.8.4.

```
\rotatebox{-90}{
  \includegraphics[width=8cm]{figure.pdf}
}
```

結果 3.8.4.



このように、`\rotatebox{-90}` を使うことにより、図を -90 度回転させることができる。

■この節のまとめ

- $\text{\LaTeX}$ の文章には、PDF 形式などの図を貼り込める。
- 実際に図を貼り込むには雛型を使う。
- 貼り込む図を回転させるには `\rotatebox{角度}{}` を使う。

3.9 参照と参考文献

3.9.1 参照

$\text{\LaTeX}$ には、数式や表、図に付いた番号を簡単に参照する機能が用意されている。まずは具体例から見ていこう。

例 3.9.1.

媒質中を物質が通過するとき、チェレンコフ光が発生するための条件は

```
\begin{equation}
  n > \frac{1}{\beta} = \sqrt{1 + \left(\frac{m}{p}\right)^2}
\end{equation}
```

である。ただし式 (1) において、

n は媒質の屈折率、 m は物質の質量、 p は物質の運動量である。

この清書結果は次のようになる。

結果 3.9.1.

媒質中を物質が通過するとき、チェレンコフ光が発生するための条件は

$$n > \frac{1}{\beta} = \sqrt{1 + \left(\frac{m}{p}\right)^2} \quad (1)$$

である。ただし式 (1) において、 n は媒質の屈折率、 m は物質の質量、 p は物質の運動量である。

例題 3.9.1. ではチェレンコフ光の発生条件の式を本文中に「式 (1)」とハードコーディングしている。

さて、**例題 3.9.1.** の式の前に別の式を挿入することになったとする。すると、式の番号はすべて 1 つずつ繰り下がるので、文章中で「式 (1)」と書いているところをすべて「式 (2)」に書き直さなければならない。このような番号の付け替えを手で行うのはとても大変であるし、間違いが起りやすい。

こういった手間を省くために、 $\text{\LaTeX}$ には数式や表、図に名前を割り当てる機能が用意されている。

例 3.9.2.

媒質中を物質が通過するとき、チェレンコフ光が発生するための条件は

```
\begin{equation}
  n > \frac{1}{\beta} =
  \sqrt{1+\left(\frac{m}{p}\right)^2} \label{cherenkov}
\end{equation}
```

である。ただし式 (`\ref{cherenkov}`) において、

`\(n\)` は媒質の屈折率、`\(m\)` は物質の質量、`\(p\)` は物質の運動量である。

例題 3.9.2. は**例題 3.9.1.** と同じ清書結果を与える。

数学環境中で `\label` を使うとその数式に名前が付けられる。付ける名前は `\label` の後の `{ }` の中に記述する。**例題 3.9.2.** では `cherenkov` という名前が付けられている。

さて、名前の付けられた数式をその名前で参照すると、その箇所が数式の番号に展開される。参照は `\ref` コマンドを使う。**例題 3.9.2.** では `\ref{cherenkov}` として参照している。`\label{cherenkov}` によって名前付けした数式の番号は 1 だったので、`\ref{cherenkov}` の参照によってこれが 1 に展開されることになる。

以下に参照の例を挙げておく。

例 3.9.3.

本年度の遠足について、以下のとおり決定いたしましたのでお知らせします。

```
\begin{table}
  \centering
  \caption{遠足の目的地 (学年別)}
  \label{destination}
  \begin{tabular}{|l|l|l|}
    \hline
    学年 & 目的地 & \\
    \hline \hline
    4 年生 & 近くにある公園 & \\
    \hline
    5 年生 & 遠くにある公園 & \\
    \hline
    6 年生 & かなり遠くにある公園 & \\
    \hline
  \end{tabular}
\end{table}
```

表 `\ref{destination}` をよく読んで自分の目的地をきちんと確認し、遠足の目的地に相応しい準備をするように心がけてください。

なお、おやつ持参は

このアルゴリズムは、BELLE 実験 [2] で使用するプログラムの内部で利用されている。

...

参考文献

[1] Walter Gautschi, Comm ACM. **12** 635, (1969)

[2] The BELLE Collaboration, KEK Report 95-1 (1995)

文章中に書く `\cite` では引用する論文の名前を指定する。論文の名前は自分が覚えやすいものであれば何でも構わないが、`\bibitem` によって名前が定義されている必要がある。`\bibitem` は `\begin{thebibliography}` と `\end{thebibliography}` とで囲まれた領域に書く。この領域は清書すると参考文献の章となる。`\bibitem` に続く `{ }` の中には自分で決めた論文名を書く。この名前が `\cite` によって参照されることになる。その後あとには、その論文の著者名や収録されている雑誌名などを書く。ここに書いたことは、参考文献の章が清書されるときにそのまま印刷される^{*5}。

`\cite` と `\ref` を、`\bibitem` と `\label` をそれぞれ対応付けて覚えておこう。

3.9.3 コンパイル

参照を含むソースファイルを $\text{\LaTeX}$ で処理するときは、`uplatex` コマンドを何回か実行する必要がある。これは、`uplatex` が参照関係を解決することが、1 回のコマンド実行だけではできないからである。参照が解決できていない状態では、展開されるべき参照の部分が `??` と表示される。`uplatex` の実行を何回か繰り返し、`??` になっている参照がなくなったことを確認する必要がある。

3.9.4 latexmk の利用

$\text{\LaTeX}$ を何回コンパイルする必要があるかは状況によって異なるので、自分の手で管理するのは手間がかかって仕方がない。この問題を解決する `latexmk` というツールを紹介する。

`latexmk` が動作するためには `.latexmkrc` という名前の設定ファイルが適切な場所に配置されている必要がある^{*6}。グローバルに設定を行う方法もあるが、今回は別の方法を紹介する。

以下の 2 ファイルをそれぞれ `example.tex` と `.latexmkrc` という名前で同じディレクトリに保存しよう。

例 3.9.5.

```
\documentclass[uplatex,dvipdfmx]{jsarticle}
\usepackage{hyperref}

\begin{document}
\hypertarget{A}{1 行目}はここ

\hyperlink{A}{1 行目}にジャンプ
\end{document}
```

例 3.9.6.

```
#!/usr/bin/env perl
@default_files = ('example.tex');
$latex         = 'uplatex -synctex=1 -halt-on-error';
$latex_silent  = 'uplatex -synctex=1 -halt-on-error -interaction=batchmode';
```

^{*5} 項目が増えてくると直接ファイルに書き込むのが大変になってくるだろう。詳しいやり方は説明しないが、そういった場合は `.bib` ファイルを使うとより管理しやすくなる。

^{*6} 設定ファイルの名前は、`latexmkrc` (先頭に `.` (ピリオド) なし) でもよい。

```

$bibtex          = 'upbibtex';
$dvipdf          = 'dvipdfmx %O -o %D %S';
$makeindex       = 'mendex %O -o %D %S';
$max_repeat      = 5;
$pdf_mode        = 3;
$pvc_view_file_via_temporary = 0;

```

ここまで準備ができれば、`latexmk` と入力する。これだけで必要な回数だけコンパイルし、PDF に変換するところまで全部まとめてやってくれる。複数回コンパイルを必要とするハイパーリンクがきちんと機能しているのわかるだろうか？

■この節のまとめ

- 数式や表、図には `\label{名前}` で名前を付けられる。
- 数式などに付けられた番号は、`\label{名前}` で付けた名前を手掛かりにして `\ref{名前}` と書くことで参照できる。
- 参考文献は `\cite{論文名}` のようにして参照する。
- 参考にした論文は、`\begin{thebibliography}` から `\end{thebibliography}` までの領域に書く。
- 論文は `\bibitem{論文名}` 著者、雑誌名 ... のように列挙する。
- 参照を含むソースファイルに対しては、何回か `uplatex` コマンドを実行する必要がある。これを自動化するツールが `latexmk` である。

3.10 L^AT_EX の種類について

この文書で使用した `uplatex` 以外にも、日本語が組版できる L^AT_EX として `platex` と `lualatex` がある。`platex` は `uplatex` の下位互換であり、`uplatex` が利用可能であるならば使う理由はない。`lualatex` は規格の観点において `uplatex` より優れているとも言われているので、最後に少しだけ `lualatex` の記法について説明しておく。

例 3.1.1. を `lualatex` の記法に直すと以下ようになる。

例 3.10.1.

```

\documentclass{ltjsarticle}
\begin{document}
\(\alpha\) のまわりのテイラー展開
\begin{equation}
f(x) = \sum_{n=0}^{\infty} \frac{f^{(n)}(a)}{n!} (x-a)^n
\end{equation}
\end{document}

```

`jsarticle` が `ltjsarticle` になったこと、そして `\documentclass` に与えていたオプションがなくなったことに注意しよう。

コンパイルは以下のように行う。

```
$ lualatex ex.tex
```

今回は直接 PDF が出力されるので `dvipdfmx` を使う必要はない*7。他の例題も同様の書き換えによりコンパイルできるだろう。

*7 よって、`.tex` ファイルに `[dvipdfmx]` の文言を書いてはならない。